Hardening MCP-Enabled Agentic Workflows: a Type-and-Effect System for Trace Safety

Marco Aldinucci, Andrea Bracciali, Iacopo Colonnelli, Doriana Medić, Alberto Mulone, Luca Roversi, Marco Edoardo Santimaria

Computer Science Department, University of Torino, Italy

Abstract

LLM-based hosts can invoke workflow-facing tools through the Model Context Protocol (MCP), allowing a model-mediated actor to inspect artefacts, propose changes, submit executions, and expose data across distributed localities. Such compositions create risks that per-call authorisation does not capture: execution without the exact approved proposal, capability amplification, omitted provenance, and source-insensitive information flow.

HARD-KLAIM extends the KLAIM coordination model with a locality-aware type-and-effect system and an operational semantics for such a boundary. An *agentic delegate* combines an LLM, an execution context, tool interfaces, a fixed declared capability set and a protocol state that evolves from proposal to commitment and execution. Immutable dynamic bindings, typed tool effects, exact proposal identifiers, prefix-sensitive admissibility, and trace-derived protocol stages connect proposal, approval, commitment, execution, and exposure without duplicating control state.

One-step provenance-and-authority confinement and, by induction, finite trace safety are proved. Every reachable configuration satisfying the initial and administrative-closure conditions preserves deterministic monotone stages, single assignment and unique production, conservative provenance, exact proposal–approval–commitment binding, capability non-amplification, source-closed exposure, and boundary fidelity. The guarantee is conditional on authoritative declarations, conservative tool schemas, faithful runtime adapters, and administrative closure. It is worth remarking that such a guarantee concerns boundary traces rather than the internal reasoning of the LLM.

The reported results are supported by the complete JANJI-ROCQ formalisation in Rocq, from which the mathematical presentation is synthesised. Constructing the formal proofs exposed missing premises and intermediate results; the obtained corresponding refinements were propagated back into the HARD-KLAIM calculus.

Keywords: Agentic workflows, Model Context Protocol, Type-and-effect systems, Trace safety, Coordination languages, Delegated authority, Provenance

1. Introduction

The background. Coordination languages were originally developed to manage distributed systems where computation, data, and authority span multiple domains. A key insight emerged: interactions must not remain implicit in implementation. Instead, concepts such as localities, mobility, permissions, and communication must be treated as first-class objects, suitably modeled in the semantics, often as types. KLAIM [1] pioneered this approach for mobile agents. By combining localities, distributed tuple spaces, and type-based access control, KLAIM demonstrated that a network address represents more than a location; it serves as a point where data, processes, capabilities, and access privileges meet.

The problem. Although KLAIM worked well for the mobile agent model of the late twentieth century, today's technical landscape has changed. Modern distributed computation is driven by distributed workflows, containerised services, cloud APIs, and job schedulers. Scientific workflows offer a prime example of this evolution. Open standards like the *Common Workflow Language* (CWL) [2] enable portable execution across workstations and HPC clusters, while runtimes such as STREAMFLOW [3] orchestrate these workflows over heterogeneous calls. Such a distributed

computation frequently involves coordination of remote APIs and container calls and constitutes a complex orchestration of remote APIs, container deployments, data movement, and security policies. These environments are becoming more complex as they are now exposed to Large Language Model (LLM)-based hosts via protocols like the *Model Context Protocol* (MCP) [4].

An LLM acts as an agentic engine within a host application, obtains workflow-facing capabilities through MCP and may invoke external tools that inspect, modify or submit workflows. The workflow itself may then execute remotely over distributed resources, for example over HPC, cloud or StreamFlow-managed resources.

From the system perspective, the actor of interest is not merely the isolated LLM model, but an *agentic delegate*, i.e. a dynamic operational compound made by: (i) the LLM, (ii) its prompt, (iii) the contextual memory, (iv) MCP tool clients, (v) workflow interfaces, and (vi) current authority. Protection-relevant actions performed by an agentic delegate are classified by their effects, including reading workflow artefacts, computing proposals, committing changes, executing workflows, or exposing derived information. Accordingly, an *agentic workflow* is a workflow whose structure or execution can be actively influenced by such an agentic delegate. Managing the boundary between KLAIM, MCP and agentic workflows needs more than traditional, per-tool access control.

For instance, consider an agentic workflow that connects to a scientific repository. Let us assume that asking to diagnose the reason for a failed run triggers an agentic delegate available in the agentic workflow. The delegate may read *private* logs, inspect an *untrusted* workflow file, draft a *patch*, execute a validation test, send a summary via email. Individually, each step appears benign. However, when combined together, these actions could result in *private data leakage*, *bypassed validation checks* or *unauthorised* execution plans. Similarly, such vulnerabilities have been properly described in *Agents of Chaos* [5], where risk stems from the composition of tool calls inside the agentic workflow, or identified as *Lethal Trifecta* [6] which combines access to private data, exposure to untrusted content, and external communication capabilities.

Contributions. Such composition risks require a proposal to become executable only through an explicit commitment that carries authority, and only when the resulting accumulated trace is admissible under the configured policy. The needed solution can neither rely on treating agentic delegates as a traditional mobile process, nor rely on fragile guardrails in LLM's prompts.

Type-based access control from KLAIM can be extended to agentic workflows by controlling workflow-locality properties, tool effect schemas, and commitments.

The types in Hard-KLAIM force localities to represent protection domains. In Hard-KLAIM tools carry ordinary input/output types together with explicit effect schemas. Given the invocation of an agentic delegate, which by design must first cross the MCP boundary, the types provide the basis to let the invocation become a typed action which associates locality-indexed effects to the current trace, and it is checked by the configured policy on the capabilities of agentic delegates. The required guarantee is the following *trace-safety property*:

> Every finite sequence of admitted agentic boundary steps, from a well-formed initial configuration, preserves complete well-formedness, deterministic protocol progress, single assignment and unique production, conservative provenance, and exact proposal–approval–commitment binding. It also preserves capability non-amplification, source-closed exposure, and boundary fidelity.

As we will see, the theorem-level invariants refine the safety questions introduced above. Exact authority binding and unique production realise commitment safety. Locality-indexed admissibility, capability non-amplification, and boundary fidelity support placement safety. Conservative provenance and source-closed exposure realise information-flow safety. Preserving all these conditions at every finite prefix yields distributed-composition safety.

The key one-step result toward the *trace-safety theorem* is *agentic provenance-and-authority confinement*. It is stronger than ordinary KLAIM locality access control because it connects the exact proposal–approval–commitment tuple, output provenance, immutable delegate capabilities, and exposure of the complete source closure. The complete theorem then follows by induction, with explicit administrative closure for the deployment facts that an abstract runtime adapter cannot forge.

Methodology. The scientific conception and motivation for extending KLAIM to Hard-KLAIM are the authors' original work. AI-assisted tools were used to improve the English presentation and to support the completion and mech-

anisation of initially hand-drafted definitions, theorem statements, and proofs. The Rocq development JANJI-ROCQ[1] served as a validation loop: proof attempts exposed missing premises and intermediate lemmas, and the resulting corrections were propagated back into the HARD-KLAIM calculus and its mathematical presentation. Comments in the JANJI-ROCQ sources identify the correspondence between the mechanised development and the definitions and results presented here.

Structure of the paper. Section 2 recalls the background on KLAIM, typed access control, scientific workflows, STREAMFLOW, and MCP-based tool integration. Section 3 defines the assumptions on and the threat model for agentic workflows. Section 5 presents HARD-KLAIM as the semantic type-and-effect framework and its abstract policy-oracle interface; a concrete filesystem-and-scheduler oracle is given in Appendix A. Section 6 presents the typing rules and safety properties for trace-level admissibility. Section 7 gives the operational semantics and trace-safety theorem at the workflow boundary, treating the underlying runtime abstractly and showing how SWIRL execution and data-transfer labels are lifted into HARD-KLAIM effects. Section 8 positions the contribution with respect to LLM agents, concurrency formalisms, information-flow control, and workflow security. Section 9 concludes by identifying future directions of interest in furhter developing the enforcement layer and its assumptions, and working on experimental assessment steps.

Gianluigi Ferrari, Giangi as we call him, ... TO BE ADDED FOR THE FINAL VERSION

2. Background

2.1. KLAIM *and access-control types*

Background. KLAIM (A Kernel Language for Agents Interaction and Mobility) was designed to make distributed interaction explicit [1]. Its basic entities are localities, tuple spaces, processes, and allocation environments. A process does not interact with an anonymous service: it performs an action at a named locality. A datum is not detached from its execution context: it is stored in a tuple space located at some locality. Mobility is not hidden in an implementation layer: it is an operation whose effect belongs to the semantics.

This locality-centred view makes access control a semantic issue. Once localities are explicit, permissions can be associated with them. A process may be allowed to read from a locality, write into it, execute there, or migrate to it. Type-based access control for KLAIM uses types as behavioural invariants: a type describes what actions a process may perform at which localities, and type checking compares this behaviour with the rights granted by the policies associated with the system [7].

Figure 1 isolates the essential mechanism. The process p is checked against locality-indexed rights. The first two actions are accepted because p has the corresponding rights at locality `lab`. The last action is rejected because p has only read access at `public`. The type is therefore not merely a classifier of values. It is a static approximation of permitted interaction.

Novel risks in agentic workflows. In an agentic workflow, the key entity to be validated is no longer the mobile code in the original KLAIM sense but an agentic delegate with all its capabilities. Nevertheless, the same methodological issue reappears: the system must know where an action takes place, which protection domain it affects, and whether the actor has the authority to perform it.

The dangerous behaviour is no longer only a single unauthorised read, write, execution, or migration. An agentic delegate may be allowed to read one artefact, propose a patch, and call a tool, while still being unauthorised to compose these actions into a committed workflow change or an external information exposure. Thus, in agentic workflows, the analogue of a KLAIM access-control type must be a type-and-effect discipline for delegated actions: it must describe what the delegate may observe, compute, propose, commit, execute, or expose across workflow localities. This is the main novel problem that HARD-KLAIM must solve beyond KLAIM: locality-indexed authority must be made explicit before distributed interaction can be safely controlled.

[1] Available at `https://github.com/LucaRoversi/JANJI-ROCQ/tree/main`

```
// Two localities.
loc lab;        // trusted laboratory locality
loc public;     // less trusted public locality

// Rights assigned to process p.
rights(p, lab)    = { read, write, exec };
rights(p, public) = { read };

// Behaviour approximated by the type of p.
p : lab.read(sample) ; lab.write(result) ; public.write(result)

// Type check.
// lab.read(sample)     is admissible: p has read at lab.
// lab.write(result)    is admissible: p has write at lab.
// public.write(result) is rejected: p has only read at public.
```

Figure 1: A minimal KLAIM-style access-control example.

2.2. LLM*-based agents and tool use*

LLM systems that access tools combine model outputs with calls to external APIs, knowledge sources, or execution tools. MRKL, ReAct, Toolformer, AutoGen, and also existing surveys of autonomous agents illustrate different variants of such an architecture [8, 9, 10, 11, 12]. For the present security model, the relevant architectural feature is that an LLM-based host may choose tool calls, provide arguments, consume returned values, and continue the interaction under delegated authority.

2.3. CWL*,* StreamFlow*, and* SWIRL

Scientific workflows make the distributed setting concrete. Examples are: CWL, which describes the workflow contract and its data dependencies [2]; StreamFlow which executes container-native workflows across heterogeneous HPC and cloud resources, where different steps may run in different administrative domains [3, 13];

SWIRL gives an operational intermediate representation of such distributed execution [14]. The relevant fragment of SWIRL is

$$
\begin{aligned}
R &::= \mathbf{0} \mid \langle l, D, e\rangle \mid (R_1 \mid R_2),\\
e &::= \mathbf{0} \mid \mu \mid e_1; e_2 \mid (e_1 \mid e_2),\\
\mu &::= \mathsf{exec}(s, F(s), \mathcal{M}(s)) \mid \mathsf{send}(A, c, l, l') \mid \mathsf{recv}(A, c, l, l').
\end{aligned}
$$

where, for instance, exec makes the execution locality of the step s explicit with $\mathcal{M}(s)$, while send and recv expose data movement between localities l and l'. Figure 2 shows the connection between a workflow dependency and a runtime placement, and an explicit inter-locality transfer.

SWIRL supplies the concrete semantics of the distributed workflow runtime. Its structural and reduction rules are defined in the semantics formalisation. Hard-KLAIM abstracts these rules as transitions $R \xrightarrow{\mu} R'$ that expose only security-relevant labels. Thus, SWIRL explains *how* the distributed workflow executes, while Hard-KLAIM determines whether the corresponding delegated execution or data movement is admissible.

2.4. MCP *and the agentic boundary*

The Model Context Protocol standardises how a language-model application reaches external tools and resources. Its tool-boundary model is defined in the specification and related documentation [4, 15]. In the setting considered here, the MCP-exposed tools are workflow-facing: they inspect logs and provenance, construct patches, select placement, submit runs, or publish outputs. MCP makes the model–tool boundary explicit, but an ordinary tool schema mainly constrains the shape of one call. It does not by itself express whether the accumulated sequence is committed, whether the execution target is admissible, or whether a generated output depends on protected sources.

```
# CWL dependency
step align:         reads.fastq -> aligned.bam
step call_variants: aligned.bam -> variants.vcf

# StreamFlow placement
M(align)         = hpc_node
M(call_variants) = secure_lab

# SWIRL-style execution
<hpc_node, {reads.fastq},
 exec(align, F(align), hpc_node) ;
 send(aligned.bam, c_align, hpc_node, secure_lab)>
|
<secure_lab, {},
 recv(aligned.bam, c_align, hpc_node, secure_lab) ;
 exec(call_variants, F(call_variants), secure_lab)>
```

Figure 2: From workflow contract to locality-aware execution.

Figure 3 illustrates the compositional gap. A principal may individually be allowed to read logs, edit workflow files, submit jobs, and send messages, while the delegate is not authorised to compose those rights into an unreviewed repair that executes at a public locality or exposes a summary derived from protected data. The safety of the next call therefore depends on the trace prefix, derived delegation stage, affected localities, and commitment evidence. Hard-KLAIM is the semantic layer that records those facts and checks them at this boundary.

3. System assumptions and location-dependent threat model

System specification. We consider an *agentic delegate* to whom some limited operational authority has been delegated (by a human principal). The agentic delegate consists of an LLM, its interaction context, the MCP client, the workflow-facing interfaces currently available to it, and the authority under which it acts. Through MCP, the agentic delegate may invoke a workflow directly as a callable capability, or may invoke workflow-control operations for inspection, patching, placement, submission, monitoring, and output handling. The invoked workflow is executed by an external runtime and may be decomposed across several localities.

Therefore, a locality is a protection domain at which data, execution rights, trust assumptions, and communication constraints are defined. Hard-KLAIM *formalises the finite traces of model-mediated actions and locality-indexed effects that crosses the* MCP*/workflow boundary*.

Trust assumptions. The LLM output, retrieved external content, proposed workflow changes, and natural-language tool descriptions are not trusted because the agentic delegate may be incorrect, influenced by prompt injection, or may compose its interfaces in an unintended way. By contrast, locality and artefact classifications, immutable workflow-version identities, delegate capabilities, policy data, approval verification keys, and sound tool-schema attestations are trusted because authoritative.

Technically, the host computes the visible-value lower bound used for model-local derivations, while tool-schemas conservatively report all protection-relevant outputs, dependencies, localities, production actions, and transfers. The runtime adapter faithfully reports the actual metadata-bound tool branch, fresh versioned output with their correct types, conservative runtime dependencies, execution locality, inter-locality transfers, and the exact association between a submitted run and the delegation context, workflow version, and immutable patch that authorised it. The runtime adapter also classifies every protection-relevant runtime event in the domain of the lifting; only events outside that domain may be treated as internal. Local access-control state, including filesystem permissions, scheduler limits, resource quotas, and mappings from the delegated principal to locality-specific identities, is also authoritative. The agentic delegate cannot modify this configuration, forge approval evidence, rebind immutable values, or bypass the

```
// Model-facing actions in one delegation context
observe(workflow_log@L_wf);
workflow_patch := call(patch_workflow,
                       workflow_log, eta_patch);
summary := derive_Report({workflow_log, workflow_patch});
call(send_report, summary, eta_mail);
p_run := propose(workflow_patch);
commit(p_run, approval);
call(submit_run, {p_run, run42}, eta_run);

// Protection-relevant interpretation
Read(workflow_log, L_wf);
Expose(workflow_log, L_wf, L_d);
Compute({workflow_log} => workflow_patch, L_d);
Compute({workflow_log, workflow_patch} => summary, L_d);
Expose(summary, L_d, L_ext);
Compute({workflow, workflow_patch} => p_run, L_d);
Commit(p_run, workflow, workflow_patch, approval, L_d);
Act(p_run, submit, workflow, workflow_patch, run42, L_rt);
```

Figure 3: A model-facing tool chain interpreted as an accumulated effect trace.

enforcement point. A runtime that lies about placement, dependencies, output identity, or transfers is outside the threat model.

Adversarial capabilities. Within the interfaces exposed to it, the agentic delegate may select tools and runtime targets, choose arguments, combine protected artefacts with untrusted inputs, propose workflow patches, request remote execution, and cause workflow output to be transferred or published. The threat model does not expect the need for any single tool to be unauthorised. Threats arise when individually available operations are composed into a trace whose cumulative effects exceed the delegated authority.

Location-dependent threats. Location-dependent threats are represented as inadmissible patterns of locality-indexed effects. Four recurrent threat patterns can be expressed as short trace fragments rather than complete executions. Each pattern isolates one reason why an otherwise plausible sequence of agentic operations must be rejected. Their shared vocabulary is fixed before the individual security questions are examined.

A locality L is a protection domain and a subscript identifies the role played by that locality in a particular example. Thus L_d is a delegate or derivation locality, L_r a runtime locality, and L_1, L_2 two possibly different locality domains. W identifies one immutable workflow version; δ identifies a proposed change to that version; p identifies the proposal that binds W to δ; and α denotes the evidence used to approve that proposal. The metavariables Y, r, and s denote protection-relevant values used as results or action arguments, while k denotes a production-action kind such as submit or execute.

The names identify the participants in a trace. Effects are used to record the steps affecting participants and externally relevant events. The effect $\mathsf{Compute}(S \Rightarrow Y, L)$ says that Y was produced at L from the finite source set S. It records dependency, not permission to act. The effect $\mathsf{Commit}(p, W, \delta, \alpha, L)$ records approval evidence for the exact proposal, workflow, and change. The effect $\mathsf{Act}(p, k, W, \delta, Y, L)$ records a production action of kind k at L, with Y as its argument. Finally, $\mathsf{Expose}(Y, L_1, L_2)$ records that the value crosses from L_1 to L_2. The operator ; orders effects from left to right. Consequently, the safety question concerns the complete ordered history, not only its last event. Four security questions of interest concern missing commitment, inadmissible placement, unauthorised information flow, and unsafe distributed composition.

1. *Uncommitted execution.* A delegate may move from observation or planning to a production-side action such as

$$\mathsf{Compute}(\{W, \delta\} \Rightarrow p, L_d)\ ;\ \mathsf{Act}(p, \mathsf{submit}, W, \delta, r, L_r)$$

Commitment safety asks whether a production action has the authority required by its preceding proposal. The delegate first constructs proposal p from workflow W and change δ at L_d, and then asks the runtime at L_r to submit it. The trace is unsafe when its earlier trace prefix contains no matching $\mathsf{Commit}(p, W, \delta, \alpha, L_c)$ in the same active delegation context. A commitment for a different proposal, workflow version, or change does not fill this gap. This case is important because it separates the ability to generate a plausible change from the authority to execute that change, preventing model output from becoming production activity merely by being passed to a submission tool.

2. *Execution at an inadmissible locality.* An $\mathsf{Act}(p, \mathsf{submit}, W, \delta, r, L_r)$ effect may be permitted at one runtime locality and rejected at another. For example, a workflow handling controlled data may be admissible on an authorised HPC partition but not on a public cloud resource. Admissibility therefore depends on the target locality's data, execution, network, and trust classifications, not only on the name of the invoked tool. Placement safety changes the question from *whether* the action was authorised to *where* it may occur. The effect should be read as one request to submit the exact committed change (p, W, δ), with argument r, at the particular protection domain L_r. Changing only L_r may therefore change the policy decision even though the proposal and tool operation remain identical. This case is important because a capability to invoke submit is not a global permission: safe delegation must retain placement, data-handling, network, and trust restrictions associated with the selected runtime.

3. *Unauthorised cross-locality flow.* An $\mathsf{Expose}(Y, L_1, L_2)$ effect is unsafe when the policy does not allow Y, or any protected source on which Y depends, to move from L_1 to L_2. This includes sending protected workflow inputs to a remote service, staging intermediate artefacts in a weaker domain, or publishing a report derived from controlled logs. Information-flow safety concerns values that cross a locality boundary rather than authority over execution. The effect says only that Y crosses that boundary; its security meaning also depends on the earlier computations that produced Y. If a report at L_1 was derived from a protected log, moving the report to L_2 must be checked as a flow of both the report and its recorded source. This case is important because otherwise an agent could bypass a flow restriction by transforming protected information into a new value before exporting it. Source-sensitive checking prevents such a transformation from erasing the relevant protection history.

4. *Unsafe distributed composition.* A distributed workflow may generate a trace such as

$$\mathsf{Act}(p_1, \mathsf{execute}, W_1, \delta_1, s_1, L_1)\ ;\ \mathsf{Expose}(Y, L_1, L_2)\ ;\ \mathsf{Act}(p_2, \mathsf{execute}, W_2, \delta_2, s_2, L_2),$$

where each endpoint operation is locally available, but the complete sequence violates placement, information-flow, or commitment constraints. The policy must therefore be checked over the accumulated trace rather than independently for each call. Distributed-composition safety brings execution and information flow together and asks whether individually available steps remain safe when composed. Read as a small distributed execution, the first effect runs one committed workflow change at L_1; the second transfers an intermediate value Y to L_2; and the third runs a second committed change there. The subscripts distinguish the proposals, workflows, changes, arguments, and execution domains belonging to the two actions. Even if both actions are locally permitted and the transfer is available as an interface operation, their composition may connect domains or data in a way that policy forbids. This case is important because agentic delegates select sequences of calls: checking calls separately cannot establish a property of the sequence they collectively construct. Prefix-sensitive trace checking supplies that compositional control.

A trace is rejected only when its effects are inconsistent with the declared authority, workflow state, locality classifications, or permitted flows. This is the connection with the type-and-effect discipline: effects make execution and communication explicit, locality indices identify the affected protection domains, and prefix-sensitive policy admissibility constrains their composition.

4. Motivating Examples

Two examples, in this section, illustrate exact commitment and source-sensitive flow. Remote execution and external tools are not unsafe per se; safety depends on the accumulated, locality-indexed trace.

```
OK-TRACE(q_ok)

1. observe(run_logs@HPC_GPU)
   ! Read(run_logs, HPC_GPU);
     Expose(run_logs, HPC_GPU, Workspace)

2. diagnosis := call(diagnose_local,
                     run_logs, eta_diag)
   ! Read(run_logs, HPC_GPU);
     Compute({run_logs} => diagnosis, HPC_GPU)

3. resource_patch := call(patch_resources,
                     {workflow, diagnosis}, eta_patch)
   ! Read(workflow, Workspace);
     Expose(diagnosis, HPC_GPU, Workspace);
     Compute({workflow, diagnosis}
             => resource_patch, Workspace)

4. p_res := propose(resource_patch)
   ! Compute({workflow, resource_patch}
             => p_res, Workspace)

5. commit(p_res, approval_token)
   ! Commit(p_res, workflow, resource_patch,
            approval_token, Workspace)

6. call(run_streamflow,
        {p_res, variant_calling}, eta_run)
   ! Act(p_res, submit, workflow, resource_patch,
         variant_calling, HPC_GPU)
```

Figure 4: `OK-TRACE`: an admissible AI4Science repair trace.

4.1. Example: AI4Science Tools

Consider a bioinformatics workflow specified in CWL and executed by StreamFlow over HPC and cloud resources. An LLM-based delegate reaches workflow-facing tools through MCP, inspects logs, constructs an immutable patch value, requests proposal-specific approval, and submits a run. Reading and computing remain non-production effects. A production action must carry the same proposal, workflow version, and patch that were committed.

4.1.1. An accepted trace

Figure 4 shows an admissible repair. The delegate locality is `Workspace`; metadata arguments fix the resolved tool and runtime choices before invocation.

The first observation records the transfer required to place the log contents in the model context. The next two calls bind fresh, immutable values `diagnosis` and `resource_patch` in $\Delta(q_{ok})$. The proposal action binds the fresh identifier `p_res` with type $\mathsf{Proposal}(\mathsf{workflow}, \mathsf{resource\_patch})$. Approval evidence is typed for the exact tuple $(a, q_{ok}, p_{res}, \mathsf{workflow}, \mathsf{resource\_patch})$. Only then may the submission produce

$$\mathsf{Act}(p_{res}, \mathsf{submit}, \mathsf{workflow}, \mathsf{resource\_patch}, \mathsf{variant\_calling}, \mathsf{HPC\_GPU}).$$

The exact patch is part of the effect, not recovered by an auxiliary attestation predicate.

The monotone stage discipline is

$$\mathsf{observe} \longrightarrow \mathsf{propose} \longrightarrow \mathsf{commit} \longrightarrow \mathsf{execute}. \tag{1}$$

```
NOT-OK-TRACE(q_bad)

1. observe(run_logs@HPC_GPU)
   ! Read(run_logs, HPC_GPU);
     Expose(run_logs, HPC_GPU, Workspace)

2. diagnosis := call(diagnose_remote,
                     run_logs, eta_remote)
   ! Read(run_logs, HPC_GPU);
     Expose(run_logs, HPC_GPU, LLM_Remote);
     Compute({run_logs} => diagnosis, LLM_Remote)

3. resource_patch := call(patch_cwl,
                     {workflow, diagnosis}, eta_patch)
   ! Read(workflow, Workspace);
     Expose(diagnosis, LLM_Remote, Workspace);
     Compute({workflow, diagnosis}
             => resource_patch, Workspace)

4. p_bad := propose(resource_patch)
   ! Compute({workflow, resource_patch}
             => p_bad, Workspace)

5. call(run_streamflow,
        {p_bad, variant_calling}, eta_cloud)
   ! Act(p_bad, submit, workflow, resource_patch,
         variant_calling, Cloud_GPU)
```

Figure 5: NOT-OK-TRACE: a rejected AI4Science repair trace.

Non-production tool calls are admitted in observe or propose; a call that contains an Act is admitted only in commit. Appending that effect makes the context's derived stage execute. Appendix B gives the complete derivation.

4.1.2. A rejected trace

The example in Figure 5 differs in three relevant ways. First, the remote diagnostic call exposes protected logs to LLM_Remote, which may violate the flow policy. Second, no exact Commit(p_{bad}, workflow, resource_patch, α, L) precedes the final action, and consequently the context is still in the derived stage propose, and not in the stage commit. Third, the action targets Cloud_GPU, which may be inadmissible for this workflow. Even under a counterfactually permissive remote-flow policy, the stage, commitment, and target-locality premises reject the final call.

4.2. Example: Location-sensitive workflow output exposure

A second delegate operates at SecureWorkspace. It imports a public issue, observes protected execution data, invokes a diagnostic workflow, and attempts to publish the resulting report. The report is a fresh output bound by the diagnostic call, not a mutable static result slot.

The first four actions may be individually admissible. The diagnostic schema binds `diagnostic_report` and records

$$\mathsf{Compute}(\{\mathsf{public\_issue}, \mathsf{run\_log}, \mathsf{dataset\_manifest}\} \Rightarrow \mathsf{diagnostic\_report}, \mathsf{SecureWorkspace}).$$

Hence

$$\mathsf{run\_log}, \mathsf{dataset\_manifest} \in \mathsf{sources}_E(\mathsf{diagnostic\_report}).$$

The final exposure induces a flow obligation for every source in that closure. Publication is rejected when either protected workflow source may not flow to PublicRepo. The same report may remain in the secure workspace or move to an authorised protection domain.

```
NOT-OK-FLOW(q_leak)

1. call(import_issue, public_issue, eta_import)
   ! Read(public_issue, PublicRepo);
     Expose(public_issue, PublicRepo, SecureWorkspace)

2. observe(run_log@SecureWorkspace)
   ! Read(run_log, SecureWorkspace)

3. observe(dataset_manifest@SecureWorkspace)
   ! Read(dataset_manifest, SecureWorkspace)

4. diagnostic_report := call(run_diagnostic_workflow,
      {public_issue, run_log, dataset_manifest}, eta_diag)
   ! Compute({public_issue, run_log, dataset_manifest}
            => diagnostic_report, SecureWorkspace)

5. call(publish_workflow_report,
        diagnostic_report, eta_publish)
   ! Expose(diagnostic_report,
           SecureWorkspace, PublicRepo)
```

Figure 6: `NOT-OK-FLOW`: a rejected workflow-output flow.

5. HARD-KLAIM: A Three-Level Type-and-Effect System

HARD-KLAIM provides us with a compact type-and-effect calculus for hardened agentic workflow delegation. Such a calculus allows us to compactly model the behaviour of the system and represent attack scenarios like the motivating traces discussed in the previous section. HARD-KLAIM is not a workflow language, a replacement for CWL or STREAMFLOW, or an alternative to MCP. Nor is it a general calculus of agentic AI. It is an intermediate semantic layer between an LLM-based agentic delegate, the MCP tool boundary, and the workflow runtime. Its purpose is to make explicit what the delegate may observe, what it may propose, what may be committed, and which committed actions may cross the boundary into workflow execution or external communication.

The calculus is designed around commitment safety, placement safety, information-flow safety, and distributed-composition safety. Its effects distinguish observation and computation from commitment, production-side action, and cross-locality flow. The locality indices are semantically relevant: the same workflow action may be admissible at one protection domain and rejected at another, and a sequence of individually available calls may be rejected because its accumulated effects cross an unauthorised placement or information-flow boundary.

5.1. Three security concerns

HARD-KLAIM separates three security concerns that together define the security boundary for model-mediated workflow actions. They are not separate enforcement systems, but three components of one judgemental structure. We introduce these concerns here and detail them in the following sections.

1. Static interface. The authoritative environment Γ classifies the entities against which delegated actions are checked. Its generic declaration judgement is

$$\Gamma \vdash n : D.$$

Elements of the computation, like concrete name classes, declaration forms, and tool schemas make this authoritative interface explicit.

Table 1: Type, effect, and auxiliary categories used by HARD-KLAIM.

Category	Notation	Role
Declaration descriptor	$D \in \{\mathsf{Loc}[\cdots], \tau, \mathsf{Del}[\kappa], \mathsf{Tool}[\cdots]\}$	Classifies a name in the authoritative static interface Γ.
Value type	$\tau \in \mathsf{Type}$	Classifies an immutable static or run-time value; τ ranges over deployment-specific base types.
Capability set	$\kappa \subseteq \mathsf{Cap}$	Gives the static upper bound on operations available to a delegate.
Primitive effect	$\epsilon \in \mathsf{Eff}$	Records one security-relevant boundary event: read, computation, commitment, production action, or exposure.
Effect fragment	$F \in \mathsf{Trace}(\mathsf{Eff})$	Finite sequence of primitive effects used to describe one possible tool or run-time behaviour.
Context-labelled trace	$E = (q_1, \epsilon_1) ; \cdots ; (q_n, \epsilon_n)$	Accumulated protocol, provenance, and security history, separated by delegation context.
Effect schema	$\widehat{E_t}$	Maps an exact tool invocation to a finite set of possible unlabelled effect fragments.
Auxiliary domains	$q \in \mathsf{Ctx},\ \eta \in \mathsf{Meta}$	Context identifiers and complete deployment-specific invocation metadata.

2. *Effectful delegation.* Run-time interaction is described by a judgement like

$$\Gamma; \Pi; \Delta; E \vdash a@L_d[q] \triangleright M!E' \Rightarrow \Delta'.$$

This judgement records the effects produced by a model-mediated action or action sequence and the fresh run-time bindings that it introduces. Here, the delegate a operates from locality L_d in context q; M is an atomic model-mediated action or finite action sequence; E' is the context-labelled trace fragment it produces; and Δ' is the resulting dynamic environment, updated from Δ, and Π is used to validate the admissibility of such a behaviour (A complete formalisation of the type system is presented in the next parts of this section).

Protocol progress is derived from the dynamic environment and the accumulated trace, rather than stored separately. Definitions in the following of this paper make this dependence explicit by deriving the execution stage from persistent bindings and accumulated effects.

3. *Policy admissibility.* A derivation is accepted only when the extended trace satisfies

$$\mathsf{Adm}_{\Gamma,\Pi,a}(E\,;E').$$

Admissibility is prefix-sensitive: each new effect is checked against the complete security-relevant history that precedes it. The policy oracle supplies deployment decisions, and admissibility lifts those decisions to complete traces.

The three concerns above have distinct roles: Γ and Δ classify static and run-time entities, E records what delegated interaction has done, and Π determines whether the resulting accumulated behaviour is admissible. The guarantee is relative to an authoritative configuration: if that configuration correctly describes the deployment, accepted traces respect it.

Table 1 summarises the calculus's syntactic categories. In particular, declaration descriptors and value types classify entities, whereas effects record security-relevant events in the accumulated trace; an effect schema is the effect component of a typed tool declaration, not an "effect type."

5.2. Level 1: Static interface

The *static environment* Γ contains the authoritative interface of the agentic workflow system. Its declarations are supplied, validated, or attested by workflow owners, data stewards, platform administrators, security officers, gateways, or trusted runtime components. These are external inputs that the language model does not determine. The

environment is fixed for a delegation derivation: fresh run-time names are inserted into the dynamic typing environment Δ, not into Γ.

Let the core classes of names be

$$\begin{aligned} L &\in \mathsf{LocName} && \text{(locality names)},\\ a &\in \mathsf{DelName} && \text{(delegate names)},\\ t &\in \mathsf{ToolName} && \text{(tool names)},\\ v &\in \mathsf{ValName} && \text{(protection-relevant value names)},\\ W &\in \mathsf{WfName} \subseteq \mathsf{ValName} && \text{(workflow-version names)}. \end{aligned}$$

Contexts $q \in \mathsf{Ctx}$ identify independent delegation runs. They are neither values nor declarations in Γ; they index run-time bindings and effects. The name classes do not prescribe concrete representations of files, jobs, queues, datasets, workflow runs, or MCP tools. They only distinguish semantic roles required by the rules.

The static interface is the finite typed declaration environment

$$\Gamma ::= \emptyset \mid \Gamma, L : \mathsf{Loc}[\cdots] \mid \Gamma, v : \tau \mid \Gamma, a : \mathsf{Del}[\kappa] \mid \Gamma, t : \mathsf{Tool}[L_t, \tau_{in}, \tau_{out}, \widehat{E_t}] \ .$$

A workflow declaration $\Gamma, W : \mathsf{Wf}[\cdots]$ is the distinguished case of a value declaration in which $W \in \mathsf{WfName}$. The subset relation allows an immutable workflow version either to be declared statically or to be returned freshly into Δ.

We write D for a declaration descriptor on the right of a binding in Γ: a locality descriptor $\mathsf{Loc}[\cdots]$, a value type τ, a delegate descriptor $\mathsf{Del}[\kappa]$, or a tool descriptor $\mathsf{Tool}[L_t, \tau_{in}, \tau_{out}, \widehat{E_t}]$. The generic judgement

$$\Gamma \vdash n : D$$

denotes lookup and well-formedness in this environment. The notation τ is reserved for value types, so value lookup has the specialised form $\Gamma \vdash v : \tau$. For instance, the static interface may declare that a locality is an HPC partition with restricted network exposure, that a value is a controlled log or resource request, that a workflow is in a validated state, or that a tool has an attested effect schema.

Localities. A locality is a named protection domain: for example an HPC partition, Kubernetes namespace, data enclave, repository, scheduler endpoint, MCP server, or remote inference service. The calculus only requires such a domain to be declared before it appears in an effect. We therefore keep its policy metadata abstract:

Definition 1 (Locality declaration). *A locality declaration has the form*

$$\Gamma \vdash L : \mathsf{Loc}[\chi_L],$$

where χ_L is an authoritative descriptor interpreted by the configured policy oracle. A concrete deployment may place in χ_L attributes such as locality kind, data class, execution mode, network exposure, trust level, or scheduler capabilities.

The core typing and operational rules do not inspect the internal structure of χ_L. They use the locality name in effects and delegate deployment-specific questions about the descriptor to O_Π, which is the oracle O defined by the security policy Π (Section 5.4).

Protection-relevant values. Artefacts are workflow-relevant data objects: datasets, logs, CWL files, configuration files, manifests, provenance records, reports, intermediate outputs, or generated summaries. Resources include budgets, queues, GPU partitions, memory allocations, storage quotas, and similar execution assets. They do not require separate syntactic name universes. Both are immutable protection-relevant values, used by protection declarionts.

Definition 2 (Protection declaration). *A protection declaration has the form*

$$\Gamma \vdash v : \tau.$$

The type distinguishes a controlled log, public report, HPC budget, GPU allocation request, or other deployment object.

Workflows. A workflow is treated as an evolvable object rather than merely as a file.

Definition 3 (Workflow declaration). *Let τ_{in} and τ_{out} be workflow interfaces, π a worflow policy, σ the worflow current lifecycle state, and e the worflow's evolution mode. Then*

$$\Gamma \vdash W : \mathsf{Wf}[\tau_{in}, \tau_{out}, \pi, \sigma, e],$$

is a workflow declaration. The particular state and evolution-mode vocabularies are deployment descriptors consumed by the policy oracle rather than additional syntax of the core calculus.

For example,

```
VariantCalling :
  Wf[FASTQ, VCF, clinical_policy, validated, human_approved]
```

declares a validated workflow whose policy requires human approval before a proposed evolution becomes executable. The core calculus only relies on the fact that the workflow and its policy-relevant metadata are authoritative in Γ. Workflow names denote immutable versions. A persistent update therefore produces a fresh workflow-version name in Δ, rather than mutating the declaration of W or its lifecycle state in place. This keeps Γ fixed and makes every committed change refer to a stable object.

Delegates. The controlled subject is not the language model in isolation. It is an agentic delegate: the model-mediated operational actor formed by a host application, an authority context, an MCP tool interface, and the set of workflow artefacts and runtime services currently reachable through that interface.

The *static* descriptor assigned to a delegate constrains the capabilities that it may exercise. Deployment roles, when used, are metadata interpreted by the configured policy oracle; no core rule inspects them, so they are not a component of the delegate declaration. Runtime progress is not part of this descriptor.

Definition 4 (Agentic delegate declaration). *Let the production-action kinds and capability universe be*

$$\begin{aligned} \mathsf{ActKind} &= \{\mathsf{submit}, \mathsf{execute}, \mathsf{update}, \mathsf{allocate}\}, \\ \mathsf{Cap} &= \{\mathsf{read}, \mathsf{derive}, \mathsf{propose}, \mathsf{commit}, \mathsf{expose}\} \\ &\quad \cup \{\mathsf{act}(k) \mid k \in \mathsf{ActKind}\}. \end{aligned}$$

The declaration of an agentic delegate a has the following form

$$\Gamma \vdash a : \mathsf{Del}[\kappa],$$

where $\kappa \subseteq \mathsf{Cap}$. The constructor $\mathsf{act}(k)$ reuses the action kind carried by Act*, rather than duplicating each production kind as a second capability vocabulary.*

Definition 5 (Agentic delegate protocol). *The protocol followed by an agentic delegate has the following four monotonic stages:*

$$q : \mathsf{observe} \longrightarrow q : \mathsf{propose} \longrightarrow q : \mathsf{commit} \longrightarrow q : \mathsf{execute}.$$

The above ordering is reflexive and local to a context.

In stage observe, a context may inspect permitted objects and perform non-production computations. In stage propose, it may continue such observation and non-production analysis while refining and naming candidate changes. In stage commit, one concrete proposal has been authorised. In stage execute, the corresponding production action and its run-time steps are being performed.

Let $s \preceq s'$denote the order defined in the above definition. Different contexts may progress independently. An explicit phase environment is nevertheless unnecessary because each context's current stage is derived from its persistent proposal binding and trace.

Typed tools with effect schemas. A workflow-facing tool is represented as a function with locality-indexed effects. Since its behaviour may depend on inputs, output names, and resolved run-time metadata, the effect component is a finite effect schema $\widehat{E}_t$.

Definition 6 (Typed tools with effect schemas declaration). *A typed tool with effect schemas appears in a judgement as follows:*

$$\Gamma \vdash t@L_t : \tau_{in} \to \tau_{out}!\widehat{E}_t. \tag{2}$$

The locality L_t is the locality at which the tool is hosted or mediated. For a value type τ, $\mathsf{Val}(\tau)$ *is the set of values having that type, and* $\mathsf{Name}(\tau)$ *is the set of tuples of pairwise-distinct names having its result shape, with* $\mathsf{Name}(\mathsf{Unit}) = \{()\}$. *Tool result types are ordinary data types, written* $\mathsf{data}(\tau)$*; they exclude the protocol witnesses* Proposal *and* Approval. *Proposals are introduced only by* T-Propose*, while approval evidence is accepted only through authoritative verification. Let* Meta *be the deployment-specific set of complete invocation metadata,* Eff *the set of primitive effects, and* $\mathsf{Trace}(\mathsf{Eff})$ *the set of finite unlabelled sequences over* Eff. *The schema has type*

$$\widehat{E}_t : \mathsf{Val}(\tau_{in}) \times \mathsf{Name}(\tau_{out}) \times \mathsf{Meta} \longrightarrow \mathcal{P}_{\mathsf{fin}}(\mathsf{Trace}(\mathsf{Eff})).$$

The notation $\mathcal{P}_{\mathsf{fin}}(X)$ denotes the finite subsets of X. An element $F \in \widehat{E}_t(x, \bar{y}, \eta)$ is a possible unlabelled, protection-relevant trace for input x, fresh result names $\bar{y}$, and metadata η.

The metadata η contains all choices that can affect the trace, such as the selected backend, resolved artefact identifiers, queue, container image, target locality, and workflow run. These choices therefore occur in the typed call itself rather than being selected after checking. [2]

A tool declaration is admitted only with a trusted schema attestation:

$$\mathsf{SchemaOK}_\Gamma(t).$$

This attestation asserts that the schema is finite and well formed; its output values have the declared type τ_{out} and every branch binds exactly its declared outputs; all protection-relevant reads, dependencies, and movements are represented conservatively; every Act label identifies the actual operation kind, workflow version, immutable change, payload, and target locality; and every concrete protection-relevant behaviour of the implementation for the same $(x, \bar{y}, \eta)$ is a member of the schema. If returned content is decoded at the delegate locality, its transfer to that locality must appear as an Expose effect; a returned opaque handle need not be treated as the underlying content. A natural-language description or an ordinary JSON argument schema is not such an attestation.

As a minimal illustration, `dataset` is an input variable and `view` a fresh output name:

```
view := read_dataset(dataset, eta) @ MCP_Data
  ! Read(dataset, DataEnclave);
    Compute({dataset} => view, DataEnclave)
```

Definition 7 (Well-formed tool declaration). *A well-formed tool declaration has the form*

$$\Gamma \vdash t : \mathsf{Tool}[L_t, \tau_{in}, \tau_{out}, \widehat{E}_t], \qquad \mathsf{data}(\tau_{out}), \qquad \mathsf{SchemaOK}_\Gamma(t).$$

The framework is intentionally relative to this authoritative configuration. It does not ask the language model to decide which data are sensitive, which tools are safe, which localities are trusted, or which workflow changes are scientifically admissible. Workflow owners may classify workflow changes; data stewards classify protected values and allowed flows; platform administrators define localities; security officers define exposure and commitment rules; and gateways or runtime components attest tool-effect schemas. These facts are recorded in Γ and Π, and Hard-KLAIM checks model-mediated traces against them.

[2] Technically, we write $\mathsf{binds}^{=}(F, \bar{y})$ when every non-unit result name in $\bar{y}$ is produced exactly once as the target of a Compute effect in F, and every Compute target in F is one of those result names. The reverse implication is essential: without it a schema branch could report an additional produced value that has no corresponding typed output binding. For the protection-relevant names $\mathsf{names}(x)$ in the input, write $\mathsf{conservative}(x, F)$ when

$$\mathsf{Compute}(S \Rightarrow y, L) \in F \quad \Longrightarrow \quad \mathsf{names}(x) \subseteq S.$$

This is a safe lower bound, not a claim that the input names are the only dependencies.

5.3. Level 2: Effectful delegation

Dynamic value environment. The dynamic part of Hard-KLAIM consists of immutable run-time values and a finite accumulated trace from which the four protocol stages (Definition 5) are derived.

Definition 8 (Dynamic value environment). *The context-indexed dynamic environment is a finite-support map*

$$\Delta : \mathsf{Ctx} \to (\mathsf{ValName} \rightharpoonup \mathsf{Type}),$$

accompanied by a finite list $A \subseteq \mathsf{Ctx}$ of active contexts with no duplicates. A binding $\Delta(q)(v) = \tau$ gives the type of a run-time value name in context q. A context is active exactly when $q \in A$.

Separating activity from bindings is necessary because an inactive context and an active context with no values both map every name to "undefined" in the functional environment. Consequently, $\mathsf{ctx}(\Delta)$ denotes A, and a well-formed state requires every context occurring in Δ to belong to A.

Dynamic bindings are *single-assignment*: a name is fresh when introduced and is never rebound. Its name therefore denotes immutable content for the lifetime of the trace. The notation $\Delta \oplus_q (\bar{v} : \bar{\tau})$ denotes a simultaneous extension by fresh names, and $\mathrm{dom}(\Delta)$ denotes the union of all dynamic domains. Exact dynamic extension is fixed by a two-way relation. If B is a finite list of triples (q, v, τ), then $\mathsf{ExactExt}(\Delta, \Delta', B)$ holds iff every name in B is absent from Δ in every context, the names in B are pairwise distinct, and, for all q, v, τ,

$$\Delta'(q)(v) = \tau \quad \Longleftrightarrow \quad \Delta(q)(v) = \tau \ \vee \ (q, v, \tau) \in B.$$

Intuitively speaking, an operational step preserves every old binding, installs exactly its declared results, and cannot introduce an unrelated value. The notation $\Delta \oplus_q (\bar{v} : \bar{\tau})$ denotes the target satisfying this relation for the corresponding result triples. The combined judgement

$$\Gamma; \Delta; q \vdash v : \tau$$

holds for an immutable static name declared in Γ, or for a dynamic binding $\Delta(q)(v) = \tau$, and extends componentwise to tuples and finite sets. Protection-relevant values generated by the model are not admitted as anonymous tool arguments: they must first be bound by T-Derive. Thus every protection-relevant component of a tool input x is a statically or dynamically named value; ordinary non-protection-relevant literals may remain inside the deployment-specific input type. For approval evidence only, the same judgement may also be discharged by the authoritative verification predicate $\mathsf{verify}_\Gamma(\alpha, a, q, p, W, \delta)$; replay is checked separately by the commit query against the trace prefix. Tool outputs, model-derived values, workflow-run outputs, and proposal identifiers are carried by Δ, not by predeclared mutable result slots.

Workflow declarations in Γ are immutable version descriptors. A tool that persistently changes a workflow returns a fresh version name and records an Act effect for the update. Thus policy-relevant lifecycle state never becomes stale through an unrecorded mutation of Γ.

The metavariables y, p, δ, α, Y range over $\mathsf{ValName}$, while W ranges over its distinguished subset WfName. Capital Y is only a role-specific metavariable for a protection-relevant value occurring in an effect; it is not another name class. Protection-relevant values include artefacts, workflow versions, proposal values, executable entities, resources, and evidence.

Model-mediated actions. The only addition needed to account for computation performed by the language model itself is a local derivation action.

Definition 9 (Model-mediated actions). *The model-mediated actions m and traces M are defined as:*

$$\begin{array}{rcl} M & ::= & m \mid m \,;\, M \\ m & ::= & \mathsf{observe}(Y@L_s) \mid y := \mathsf{derive}_\tau(S) \mid p := \mathsf{propose}(\delta) \mid \mathsf{commit}(p, \alpha) \\ & & \mid \bar{y} := \mathsf{call}(t, x, \eta) \end{array} \tag{3}$$

The source locality is explicit in $\mathsf{observe}(Y@L_s)$, and the same metadata η are used by typing and invocation. The trusted host exposes a finite conservative set $\mathsf{vis}_{\Gamma,\Delta,E}(q, L_d)$ of protection-relevant values that may influence the next model-generated value. Visible-set soundness requires this set to contain every such value, including values entering

through prompts, observations, decoded tool results, and retained context. A derivation $y := \mathsf{derive}_\tau(S)$ is admitted only when this visible set is a subset of S. The model therefore cannot omit a protection-relevant input from the provenance of a generated patch, report, or summary; implementations may safely add more sources. Opaque handles need not enter the visible set until their contents are decoded.

Observation obtains the contents of an object. Define the unlabelled observation fragment

$$\mathsf{obs}(Y, L_s, L_d) = \begin{cases} \mathsf{Read}(Y, L_s), & L_s = L_d, \\ \mathsf{Read}(Y, L_s)\ ;\ \mathsf{Expose}(Y, L_s, L_d), & L_s \neq L_d. \end{cases}$$

Thus observing a remote object records the corresponding transfer to the delegate-control locality. An implementation that returns only an opaque handle must type that handle as a separate value rather than claim that Y was observed.

Primitive effects. The five primitive effect forms are formally defined by the following definition, where Act has been further detailed to carry the exact authorised change and its kind.

Definition 10 (Primitive effects). *The set* Eff *of primitive effects are defined by the following grammar, with* $\epsilon \in \mathsf{Eff}$

$$\begin{aligned} \epsilon \quad ::= \quad & \mathsf{Read}(Y, L) \mid \mathsf{Compute}(S \Rightarrow Y, L) \mid \mathsf{Commit}(p, W, \delta, \alpha, L) \mid \\ & \mathsf{Act}(p, k, W, \delta, Y, L) \mid \mathsf{Expose}(Y, L_1, L_2). \end{aligned} \tag{4}$$

where $k \in \mathsf{ActKind}$ *range over the production-action kinds fixed in Definition 4.*

$\mathsf{Read}(Y, L)$ records an observation at L. The source-aware effect $\mathsf{Compute}(S \Rightarrow Y, L)$ records that the fresh immutable value Y is produced at L from the finite conservative dependency set S. $\mathsf{Commit}(p, W, \delta, \alpha, L)$ records authorisation of proposal p for the immutable change δ on workflow version W. A production action is $\mathsf{Act}(p, k, W, \delta, Y, L)$: it carries the same proposal, workflow version, and change syntactically, together with the operation kind, payload Y, and target locality. No separate realises predicate is required. $\mathsf{Expose}(Y, L_1, L_2)$ records movement between protection domains.

The shared proposal may authorise the one production call that moves a context from commit to execute, followed by the run-time steps of that same submitted run. It cannot authorise a second production call in the same context because calls are no longer admitted after the transition to execute. All associated run-time steps must preserve the same (q, p, W, δ) wrapper metadata.

Definition 11 (Effect trace). *A context-labelled effect trace is a finite sequence*

$$E ::= (q_1, \epsilon_1)\ ;\ \cdots\ ;\ (q_n, \epsilon_n).$$

Let $\mathsf{effects}_q(E)$ *denote the effects in context q, let* $\mathsf{effects}(E)$ *denote the unlabelled effects in E, and let* $\mathsf{lab}_q(\epsilon_1\ ;\cdots;\epsilon_n) = (q, \epsilon_1)\ ;\ \cdots\ ;\ (q, \epsilon_n)$. *A tuple* $\bar{v}$ *is fresh, written* $\bar{v}\#(\Gamma, \Delta, E)$*, when none of the names in v is declared in* Γ *or occurs in* Δ *or E.*

Next, we introduce some notation that will be useful to reason about effect traces.

Let $\mathsf{init}_\Gamma(Y, L)$ be an authoritative initial placement. Given a prefix E, we write $E \Downarrow_\Gamma Y@L$ when the initial placement holds or when the prefix contains a $\mathsf{Read}(Y, L)$, $\mathsf{Compute}(S \Rightarrow Y, L)$, or $\mathsf{Expose}(Y, L', L)$ effect.

A produced value Y is fresh at prefix E, written $\mathsf{fresh}_\Gamma(E, Y)$, when $Y \notin \mathrm{dom}(\Gamma)$, it is not initially placed at any locality, and it has not previously been the target of a Compute effect.

We write

$$\mathsf{proposed}_E(q, p, W, \delta) \quad \text{iff} \quad \exists L.\, (q, \mathsf{Compute}(\{W, \delta\} \Rightarrow p, L)) \in E.$$

For an unlabelled fragment F, $\mathsf{hasAct}(F)$ mean that F contains an Act effect. For $q \in \mathsf{ctx}(\Delta)$, the *protocol stage* is the derived function

$$\mathsf{stage}_{\Delta,E}(q) = \begin{cases} \mathsf{execute}, & \exists p, k, W, \delta, Y, L.\ \mathsf{Act}(p, k, W, \delta, Y, L) \in \mathsf{effects}_q(E), \\ \mathsf{commit}, & \exists p, W, \delta, \alpha, L.\ \mathsf{Commit}(p, W, \delta, \alpha, L) \in \mathsf{effects}_q(E), \\ \mathsf{propose}, & \exists p, W, \delta.\ \Delta(q)(p) = \mathsf{Proposal}(W, \delta), \\ \mathsf{observe}, & \text{otherwise.} \end{cases}$$

The $stage_{\Delta,E}(\_)$ is undefined for inactive contexts. This is also reflected in the typing rules (Section 6), whose premises enforce context activation.

The clauses are ordered: execution dominates commitment, which dominates a persistent proposal binding. Hence the stage cannot regress even though bindings and effects remain available. Because every schema branch will be checked against the same derived stage, a schema mixing production and non-production alternatives is deliberately rejected: such behaviours must be exposed as separate tool interfaces.

Finally, $\mathsf{req}(F)$ collects the capabilities induced by F: read for Read, expose for Expose, and $\mathsf{act}(k)$ for each production action of kind k.

Definition 12 (Dependency relation and source closure). *The direct dependency relation induced by E is*

$$X \rightsquigarrow_E Y \quad \textit{iff} \quad \exists q, L, S.\ (q, \mathsf{Compute}(S \Rightarrow Y, L)) \in E \wedge X \in S.$$

The finite source closure is

$$\mathsf{sources}_E(Y) = \{X \mid X(\rightsquigarrow_E)^* Y\}.$$

It is reflexive, so $Y \in \mathsf{sources}_E(Y)$. Over-approximating a dependency set may reject a safe exposure; omitting a true dependency violates schema soundness or the visible-set premise of model derivation.

Value types. The overall grammar we are defining embeds the delegation-specific constructors in the ordinary deployment-specific type universe rather than presenting them as the whole type language.

Definition 13 (Value types). *Let β range over deployment-specific base types, including artefact, resource, evidence-reference, and workflow-interface descriptors such as $\mathsf{Wf}[\tau_{in}, \tau_{out}, \pi, \sigma, e]$. Then, value types are defined as follows:*

$$\begin{aligned} \tau ::= \beta \mid \mathsf{Unit} \mid \tau \times \tau \mid \mathsf{Set}(\tau) \mid \mathsf{Patch}(W) \mid \\ \mathsf{Proposal}(W, \delta) \mid \mathsf{Approval}(a, q, p, W, \delta). \end{aligned} \tag{5}$$

The above grammar generates Type, the set of well-formed value types. Unit is the no-value result type, $\tau_1 \times \tau_2$ classifies paired values, and $\mathsf{Set}(\tau)$ classifies finite sets of τ-values. The parameters in $\mathsf{Patch}(W)$, $\mathsf{Proposal}(W, \delta)$, and $\mathsf{Approval}(a, q, p, W, \delta)$ are ordinary first-order tags carried by the corresponding values. They are checked by ordinary premises and oracle queries. They do not introduce dependent functions, type-level computation, equality types, or dependent pattern matching. Thus, the calculus remains a simple type-and-effect system with explicit protocol evidence.

Well-formedness of these constructors requires their indices to belong to the corresponding name domains, and it does not require indexed value names to be statically declared in Γ. In particular, a fresh workflow-version name returned into Δ may index a subsequent Patch, $\mathsf{Proposal}$, or $\mathsf{Approval}$ within the same delegation context, subject to the ordinary typing and policy premises.

We write $\mathsf{data}(\tau)$ when τ contains no occurrence of $\mathsf{Proposal}$ or $\mathsf{Approval}$. Hence, only the rule T-Propose (Section 6) can introduce a proposal-typed dynamic binding. A value of $\mathsf{Patch}(W)$ is an immutable change applicable to workflow version W. Should an update materialise in a new workflow version, that version is a fresh dynamic output of the corresponding production operation. A value of $\mathsf{Proposal}(W, \delta)$ is non-executable; commitment of that exact proposal is recorded by a Commit effect rather than by an unused result value. Approval evidence is context- and proposal-specific. The judgement

$$\Gamma; \Delta; q \vdash \alpha : \mathsf{Approval}(a, q, p, W, \delta)$$

is validated by verifying the unforgeable evidence payload against trusted keys and policy data in Γ. Consequently, an approval for another context, proposal, workflow version, or patch cannot be reused. Replay policy is part of the terminating commit query. The derived monotone stage already prevents a second commitment in one context.

Central judgement. Finally, we can define the format of a *central judgement*.

Definition 14 (Central judgement). *Let Γ be the authoritative static environment, Π the active policy, Δ the input dynamic typing environment, and E the accumulated trace preceding M. A central judgment has the following format*

$$\Gamma; \Pi; \Delta; E \vdash a@L_d[q] \triangleright M!E' \Rightarrow \Delta'. \tag{6}$$

where, the delegate a operates from locality L_d in context q; M is an atomic model-mediated action or finite action sequence; E' is the context-labelled trace fragment it produces; and Δ' is the resulting dynamic environment.

All results that may be used subsequently are explicitly named in Δ'. Every successful rule (Section 6) checks $\mathsf{Adm}_{\Gamma,\Pi,a}(E\ ;\ E')$, so the judgement is a conditional safety statement relative to the authoritative configuration rather than a claim of absolute safety for M.

5.4. Level 3: Policy admissibility

A policy instance Π fixes a terminating oracle O_Π; hence the oracle is determined by Π, rather than being a hidden argument of admissibility. Its finite query family is

$$\begin{aligned} Q ::= {} & \mathsf{observe}(a, Y, L) \mid \mathsf{propose}(a, W, \delta, L) \mid \mathsf{commit}(a, q, p, W, \delta, \alpha, L) \mid \\ & \mathsf{call}(a, t, L_d, L_t, \eta) \mid \mathsf{act}(a, k, W, \delta, Y, L) \mid \mathsf{flow}(a, Y, L_1, L_2), \end{aligned}$$

and

$$O_\Pi(\Gamma, E, Q) \in \{\mathsf{allow}, \mathsf{deny}\}.$$

Readable premises such as $O_\Pi \vdash \mathsf{canCall}(a, t, L_d, L_t, \eta)$ abbreviate a successful query. Tool lookup ranges over the well-formed declarations of Definition 7 and it already entails $\mathsf{SchemaOK}_\Gamma(t)$. The language model never answers these queries. An oracle instance may carry finite deployment metadata that is not a core name class. In particular, the filesystem-and-scheduler instance resolves a delegate to platform-specific principal identities internally.

Definition 15 (Prefix admissibility). *The closed relation $\mathsf{Adm}_{\Gamma,\Pi,a}(E)$ holds when, for every decomposition $E = E_0\ ;\ (q, \epsilon)\ ;\ E_1$, the obligation induced by ϵ is satisfied against the preceding prefix E_0:*

Read. *If $\epsilon = \mathsf{Read}(Y, L)$, then $E_0 \Downarrow_\Gamma Y@L$ and $O_\Pi(\Gamma, E_0, \mathsf{observe}(a, Y, L)) = \mathsf{allow}$.*

Compute. *If $\epsilon = \mathsf{Compute}(S \Rightarrow Y, L)$, then $\mathsf{fresh}_\Gamma(E_0, Y)$ and $E_0 \Downarrow_\Gamma X@L$ for every $X \in S$.*

Commit. *If $\epsilon = \mathsf{Commit}(p, W, \delta, \alpha, L)$, then $\mathsf{proposed}_{E_0}(q, p, W, \delta)$ and*

$$O_\Pi(\Gamma, E_0, \mathsf{commit}(a, q, p, W, \delta, \alpha, L)) = \mathsf{allow}.$$

Act. *If $\epsilon = \mathsf{Act}(p, k, W, \delta, Y, L)$, then there are α, L_c such that $(q, \mathsf{Commit}(p, W, \delta, \alpha, L_c)) \in E_0$ and*

$$O_\Pi(\Gamma, E_0, \mathsf{act}(a, k, W, \delta, Y, L)) = \mathsf{allow}.$$

Expose. *If $\epsilon = \mathsf{Expose}(Y, L_1, L_2)$, then $E_0 \Downarrow_\Gamma Y@L_1$ and, for every $B \in \mathsf{sources}_{E_0}(Y)$,*

$$O_\Pi(\Gamma, E_0, \mathsf{flow}(a, B, L_1, L_2)) = \mathsf{allow}.$$

The relation is prefix-closed by construction and makes all previously implicit parameters explicit. Proposal and call authority are checked by the respective typing rules because the corresponding Compute fragments do not identify which source action produced them.

Definition 16 (Effect order and exact commitment binding). *For a trace E, write $(q_j, \epsilon_j) \prec_E (q_k, \epsilon_k)$ when the first pair occurs earlier. Then prefix admissibility entails*

$$\begin{aligned} \mathsf{Act}(p, k, W, \delta, Y, L) \in \mathsf{effects}_q(E) \Rightarrow \exists \alpha, L_c.\ & \mathsf{Commit}(p, W, \delta, \alpha, L_c) \in \mathsf{effects}_q(E) \\ & \wedge (q, \mathsf{Commit}(p, W, \delta, \alpha, L_c)) \prec_E (q, \mathsf{Act}(p, k, W, \delta, Y, L)). \end{aligned} \tag{7}$$

The equality of p, W, δ is syntactic, and δ is immutable by single assignment. This removes both proposal substitution and time-of-check to time-of-use rebinding.

Because traces, schemas, source sets, and query families are finite, type checking and trace admissibility are decidable relative to terminating schema instantiation, evidence verification, and policy-oracle queries.

5.5. A concrete policy instantiation

Appendix A gives a complete decidable instantiation of the oracle interface for a filesystem-and-scheduler setting. It resolves delegated identities and defines observation, proposal, commitment, metadata-bound call, action-kind, and source-flow queries using finite permission and quota checks. Stronger classification or declassification policies can replace it without changing the core effect forms.

6. Judgements, Rules, and Safety Properties

For the sake of readability, we report here again the central judgement:

$$\Gamma; \Pi; \Delta; E \vdash a@L_d[q] \triangleright M!E' \Rightarrow \Delta'.$$

The prefix E and single-assignment environment Δ are dynamic inputs. The result is a new fragment E' and an updated value environment; the current protocol stage is $\mathsf{stage}_{\Delta,E}(q)$.

Readable policy premises abbreviate calls to the policy oracle. In particular, $O_\Pi \vdash \mathsf{canPropose}(a, W, \delta, L)$ and $O_\Pi \vdash \mathsf{canCall}(a, t, L_d, L_t, \eta)$ mean that the corresponding query returns allow. The remaining effect-specific queries are generated by $\mathsf{Adm}_{\Gamma,\Pi,a}$.

In the rest of this section, we first introduce the typing rules for (central) judgments and then we state properties of interest for our framework.

6.1. Typing rules

Observation. The source locality is part of the action. Observation remains admissible while a context is refining proposals, so it is permitted in both observe and propose. If the source locality differs from the delegate locality, the observation fragment contains the corresponding exposure.

$$\frac{\Gamma \vdash a : \mathsf{Del}[\kappa] \quad \mathsf{stage}_{\Delta,E}(q) \in \{\mathsf{observe}, \mathsf{propose}\} \quad \Gamma; \Delta; q \vdash Y : \tau_Y \quad F_o = \mathsf{obs}(Y, L_s, L_d) \quad \mathsf{req}(F_o) \subseteq \kappa \quad \mathsf{Adm}_{\Gamma,\Pi,a}(E\,;\mathsf{lab}_q(F_o))}{\Gamma; \Pi; \Delta; E \vdash a@L_d[q] \triangleright \mathsf{observe}(Y@L_s)!\mathsf{lab}_q(F_o) \Rightarrow \Delta} \quad \text{(T-Obs)}$$

A remote observation is therefore rejected when the flow to L_d is not permitted, even if reading at L_s is locally authorised.

Model-local derivation. A protection-relevant value generated by the model is fresh and records every value that may have influenced it. The host-maintained visible set, rather than the model, supplies the lower bound on that source set.

$$\frac{\begin{array}{c}\Gamma \vdash a : \mathsf{Del}[\kappa] \quad \mathsf{stage}_{\Delta,E}(q) \in \{\mathsf{observe}, \mathsf{propose}\} \quad y\#(\Gamma, \Delta, E) \quad \Gamma \vdash \tau\ \mathsf{type} \quad \mathsf{data}(\tau) \quad \mathsf{derive} \in \kappa \\ \forall X \in S.\ \Gamma; \Delta; q \vdash X : \tau_X \quad \mathsf{vis}_{\Gamma,\Delta,E}(q, L_d) \subseteq S \quad \mathsf{Adm}_{\Gamma,\Pi,a}(E\,;(q, \mathsf{Compute}(S \Rightarrow y, L_d)))\end{array}}{\Gamma; \Pi; \Delta; E \vdash a@L_d[q] \triangleright y := \mathsf{derive}_\tau(S) \quad !\,(q, \mathsf{Compute}(S \Rightarrow y, L_d)) \Rightarrow \Delta \oplus_q (y : \tau)} \quad \text{(T-Derive)}$$

This rule is the only formal account needed for summaries, patches, or reports computed inside the LLM host. Tool-internal computation remains described by attested tool schemas.

Non-executable proposal. Only an immutable patch value can be proposed. It may be a static versioned object or a fresh run-time result; the proposal name itself is fresh and remains non-executable.

$$\frac{\begin{array}{c}\Gamma \vdash a : \mathsf{Del}[\kappa] \quad \mathsf{stage}_{\Delta,E}(q) \in \{\mathsf{observe}, \mathsf{propose}\} \quad p\#(\Gamma, \Delta, E) \quad \Gamma; \Delta; q \vdash \delta : \mathsf{Patch}(W) \quad \mathsf{propose} \in \kappa \\ O_\Pi \vdash \mathsf{canPropose}(a, W, \delta, L_d) \quad \mathsf{Adm}_{\Gamma,\Pi,a}(E\,;(q, \mathsf{Compute}(\{W, \delta\} \Rightarrow p, L_d)))\end{array}}{\Gamma; \Pi; \Delta; E \vdash a@L_d[q] \triangleright p := \mathsf{propose}(\delta) \quad !\,(q, \mathsf{Compute}(\{W, \delta\} \Rightarrow p, L_d)) \Rightarrow \Delta \oplus_q (p : \mathsf{Proposal}(W, \delta))} \quad \text{(T-Propose)}$$

The rule cannot overwrite a previous patch or proposal binding. Consequently, subsequent equality of δ means equality of immutable content identity.

Commitment. Approval evidence is bound to the context, proposal identifier, workflow version, and immutable patch.

$$\frac{\Gamma \vdash a : \mathsf{Del}[\kappa] \quad \mathsf{stage}_{\Delta,E}(q) = \mathsf{propose} \quad \Delta(q)(p) = \mathsf{Proposal}(W,\delta) \quad \mathsf{proposed}_E(q,p,W,\delta) \\ \Gamma;\Delta;q \vdash \alpha : \mathsf{Approval}(a,q,p,W,\delta) \quad \mathsf{verify}_\Gamma(\alpha,a,q,p,W,\delta) = \mathsf{true} \quad \mathsf{commit} \in \kappa \quad \mathsf{Adm}_{\Gamma,\Pi,a}(E\,;(q,\mathsf{Commit}(p,W,\delta,\alpha,L_d)))}{\Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright \mathsf{commit}(p,\alpha) \quad !\,(q,\mathsf{Commit}(p,W,\delta,\alpha,L_d)) \Rightarrow \Delta} \tag{T-Commit}$$

The explicit verification premise is stronger than merely finding a value with an approval-shaped type. It states that the trusted verifier accepts this evidence for exactly the delegate, context, proposal, workflow version, and patch that the commitment records. The admissibility query then prevents replay at an unacceptable trace prefix.

Tool invocation. A call binds fresh result names and contains the exact metadata used to instantiate and invoke the tool. For a fragment F, let $\mathsf{pids}(F)$ collect proposal identifiers occurring in production effects, and let $\mathsf{pids}(x)$ collect identifiers explicitly carried by the input.

Definition 17 (Call-fragment guard). *The call-fragment guard is a check that is defined as:*

$$\begin{aligned}\mathsf{callGuard}_{\Delta,E,q}(F,x) \quad \textit{iff} \quad & \left(\begin{array}{l}\mathsf{hasAct}(F) \wedge \mathsf{stage}_{\Delta,E}(q) = \mathsf{commit} \\ \quad \vee \\ \neg\mathsf{hasAct}(F) \wedge \mathsf{stage}_{\Delta,E}(q) \in \{\mathsf{observe},\mathsf{propose}\}\end{array}\right) \\ & \wedge \neg\exists p,W,\delta,\alpha,L.\ \mathsf{Commit}(p,W,\delta,\alpha,L) \in F \\ & \wedge \mathsf{pids}(F) \subseteq \mathsf{pids}(x).\end{aligned}$$

$$\frac{\Gamma \vdash a : \mathsf{Del}[\kappa] \quad \Gamma \vdash t : \mathsf{Tool}[L_t,\tau_{in},\tau_{out},\widehat{E_t}] \quad \Gamma;\Delta;q \vdash x : \tau_{in} \quad \bar{y}\#(\Gamma,\Delta,E) \quad \bar{y} \in \mathsf{Name}(\tau_{out}) \quad F_t \in \widehat{E_t}(x,\bar{y},\eta) \quad O_\Pi \vdash \mathsf{canCall}(a,t,L_d,L_t,\eta) \\ \forall F \in \widehat{E_t}(x,\bar{y},\eta).\ \mathsf{Adm}_{\Gamma,\Pi,a}(E\,;\mathsf{lab}_q(F)) \wedge \mathsf{req}(F) \subseteq \kappa \wedge \mathsf{callGuard}_{\Delta,E,q}(F,x) \wedge \mathsf{binds}^{=}(F,\bar{y}) \wedge \mathsf{conservative}(x,F)}{\Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright \bar{y} := \mathsf{call}(t,x,\eta)!\mathsf{lab}_q(F_t) \quad \Rightarrow \Delta \oplus_q (\bar{y} : \tau_{out})} \tag{T-Call}$$

The exact-binding and conservative-dependency premises make the parts of the schema attestation used by preservation explicit at the call site. The former relates all and only the fresh result bindings to the reported Compute targets; the latter ensures that every protection-relevant input is included in the source set of each result computation.

The no-commit condition and proposal-identifier condition are separate conjuncts. A tool cannot manufacture a commitment, and an Act may mention only a proposal explicitly supplied in its input. Every possible schema branch is checked; the actual branch F_t is merely the member reported after the trusted invocation. The result tuple has shape τ_{out} by $\bar{y} \in \mathsf{Name}(\tau_{out})$, while $\mathsf{SchemaOK}_\Gamma(t)$ attests that the implementation returns values of that declared type. The stage premise admits non-production calls only in observe or propose, and admits a production call only in commit. Appending an Act makes the derived stage execute, so another production call is no longer admissible.

Trace sequencing.

$$\frac{\Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright m_1!E_1 \Rightarrow \Delta_1 \quad \Gamma;\Pi;\Delta_1;E\,;E_1 \vdash a@L_d[q] \triangleright m_2\,;\cdots;m_n!E_2 \Rightarrow \Delta_n}{\Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright m_1\,;\cdots;m_n!(E_1\,;E_2) \Rightarrow \Delta_n} \tag{T-Seq}$$

The second premise receives the complete trace prefix and value environment produced by the first action; its protocol stage is therefore derived from the updated pair $(\Delta_1, E\,; E_1)$.

6.2. Safety properties of interest

Typing preserves admissibility. If

$$\Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright M!E' \Rightarrow \Delta',$$

then $\mathsf{Adm}_{\Gamma,\Pi,a}(E\,;E')$, every name newly introduced in Δ' is fresh and single-assignment, and every production transition in E' originates from a call checked at derived stage commit. This follows by induction on the typing derivation. The atomic cases contain the required admissibility premise, and T-Seq applies the induction hypothesis to the complete intermediate prefix.

Proposition 1 (Deterministic and monotone stage reconstruction). *For an active context, the ordered definition of* stage *returns exactly one of* observe, propose, commit, execute. *Moreover, if* Δ' *exactly extends* Δ, $E' = E\,;E_a$, *and active contexts are preserved, then*

$$\mathsf{stage}_{\Delta,E}(q) = s \;\wedge\; \mathsf{stage}_{\Delta',E'}(q) = s' \quad\Longrightarrow\quad s \preceq s'.$$

Determinism follows by case analysis on the ordered tests for an action, a commitment, and a proposal binding. Monotonicity follows because exact extension preserves all old bindings and trace extension preserves all old effects; a later test may become true, but an earlier fact cannot disappear.

Proposition 2 (Single assignment and unique production). *Suppose a typed atomic action changes* (Δ, E) *to* (Δ', E'). *There are finite lists B of result bindings and F of new effects such that* $\mathsf{ExactExt}(\Delta, \Delta', B)$, $E' = E\,;F$, *every binding in B has exactly one producing* Compute *in F, and every producing* Compute *in F targets a binding in B. This* exact production delta *is the bridge between the typing environment and the trace: freshness alone prevents rebinding, but the two directions of the bridge are what exclude both an unrecorded binding and an unbound production event. Consequently exact extensions preserve global single assignment and give every installed dynamic name one unique producer.*

Exact context-indexed commitment safety. If E is the trace of a well-formed reachable configuration (equivalently, for the static fragment, if it is produced from a well-formed state by the typing rules), then for every context q,

$$\begin{aligned}\mathsf{Act}(p,k,W,\delta,Y,L) \in \mathsf{effects}_q(E) \Rightarrow \exists \alpha, L_c.\ (q, \mathsf{Commit}(p,W,\delta,\alpha,L_c)) \prec_E (q, \mathsf{Act}(p,k,W,\delta,Y,L)) \\ \wedge\ \Delta(q)(p) = \mathsf{Proposal}(W,\delta) \\ \wedge\ \Gamma;\Delta;q \vdash \alpha : \mathsf{Approval}(a,q,p,W,\delta) \\ \wedge\ \mathsf{verify}_\Gamma(\alpha,a,q,p,W,\delta) = \mathsf{true}.\end{aligned} \tag{8}$$

The same immutable δ occurs in proposal, approval, commitment, and action. A Compute effect alone cannot establish this property.

Source-sensitive flow safety. For a prefix ending in an exposure,

$$\mathsf{Adm}_{\Gamma,\Pi,a}(E_0\,;(q,\mathsf{Expose}(Y,L_1,L_2))) \Rightarrow \forall B \in \mathsf{sources}_{E_0}(Y).\ O_\Pi(\Gamma,E_0,\mathsf{flow}(a,B,L_1,L_2)) = \mathsf{allow}. \tag{9}$$

Tool outputs obtain their sources from sound schemas; model outputs obtain them from T-Derive's visible-set premise; and workflow-run outputs obtain them from the conservative run-time lifting defined next.

Boundary fidelity. A typed call fixes $(t, x, \bar{y}, \eta)$ before invocation. Its operational rule uses that same tuple and appends exactly the reported member of $\widehat{E}_t(x, \bar{y}, \eta)$. Thus a backend, queue, target locality, result binding, or proposal identifier cannot be changed between checking and invocation without producing a different, untyped call.

7. Operational Semantics at the Workflow Boundary

The action-and-effect classification and prefix-admissibility judgement induce a minimal boundary semantics whose invariants hold over reachable configurations. The semantics does not model the internal reasoning of the LLM or reproduce the internal semantics of the workflow engine.

7.1. Runtime abstraction and the SWIRL *instantiation*

Assume an underlying workflow runtime transition

$$R \xrightarrow{\mu} R'.$$

SWIRL is the concrete runtime instance [14]. Its published rules define distributed execution and communication. Hard-KLAIM consumes only the labels relevant to commitment, placement, provenance, and flow.

For a security-relevant runtime event, the trusted boundary adapter supplies

$$\mathsf{auth}_R(\mu) = (q, p, W, \delta), \qquad \mathsf{deps}_R(\mu) = D_\mu, \qquad \mathsf{out}_R(\mu) = \bar{z} : \bar{\tau}.$$

The first tuple identifies the submitted run and its exact immutable proposal. The set D_μ conservatively contains every protection-relevant object that may influence the outputs of the event. It includes the data inputs and, when relevant, the executable or container, parameters, environment, committed patch, and external state represented at the boundary. The output tuple gives fresh, versioned names and their correct types, as reported by the trusted boundary adapter.

The lifting of the relevant SWIRL labels is

$$\begin{aligned} \mathsf{lift}_{p,W,\delta}(\mathsf{exec}(s, F(s), L), D_\mu, \bar{z}) &= \mathsf{Act}(p, \mathsf{execute}, W, \delta, s, L) \,;\prod_{Z \in \bar{z}} \mathsf{Compute}(D_\mu \Rightarrow Z, L), \\ \mathsf{lift}_{p,W,\delta}(\mathsf{send}(A, c, L_1, L_2), D_\mu, ()) &= \mathsf{Expose}(A, L_1, L_2). \end{aligned} \tag{10}$$

The product uses any fixed order. A matching receive does not add a second exposure because the send already records the transfer. The boundary adapter is required to classify every protection-relevant runtime event in the domain of the lifting; only events that are not protection-relevant may remain outside that domain and be treated as internal.

For an initial SWIRL component $\langle L, D, e\rangle$, the authoritative environment satisfies $\mathsf{init}_\Gamma(Y, L)$ for each $Y \in D$. Initial executables and configuration objects used in a dependency set are classified in the same way. The adapter is required to version newly produced outputs rather than reuse a mutable name.

7.2. Boundary configurations and transitions

Definition 18 (Hard-KLAIM configuration). *A* Hard-KLAIM *configuration C is defined as:*

$$C ::= \langle R, a@L, \Delta, E\rangle,$$

where R is the boundary-managed tool and workflow state and the remaining components retain their established meanings.

Note that proposal and commitment state are not duplicated in a separate store: proposal bindings are in Δ, while exact commitments and protocol stage are projections of E. An implementation may cache these projections, but the cache is not a semantic state.

A strengthened boundary configuration retains the finite state required by trace preservation:

$$\widehat{C} ::= \langle C, A, H\rangle.$$

The component A is the finite, duplicate-free active-context list, and H is the finite sequence of public boundary labels. The extra components do not duplicate proposal or stage state. They retain two facts that cannot be reconstructed from the core functional environment alone: whether an empty context is active, and which trusted boundary rule introduced a dynamic name. This information is needed to state origins and exact boundary fidelity without making either an implicit meta-level assumption.

Projecting a strengthened configuration back to its core component by forgetting A and H, while retaining C, does not change the accepted traces. On every well-formed reachable state, a proposal index is reconstructed from the proposal bindings and exact Commit effects, and an explicit stage map would equal $q \mapsto \mathsf{stage}_{\Delta,E}(q)$ on $\mathsf{ctx}(\Delta)$. The transition rules therefore operate directly on the information from which those auxiliary structures would be computed.

The transition relation is parameterised by the authoritative interface and policy, which fixes its oracle:

$$\Gamma; \Pi \vdash C \xrightarrow{\lambda} C'.$$

Write $\bar{\nu}\#C$ when the names are fresh for all components of C and are not declared in Γ.

Observation.

$$\frac{\Gamma; \Pi; \Delta; E \vdash a@L_d[q] \triangleright \mathsf{observe}(Y@L_s)!E_o \Rightarrow \Delta'}{\Gamma; \Pi \vdash \langle R, a@L_d, \Delta, E\rangle \xrightarrow{\mathsf{observe}(q,Y,L_s)} \langle R, a@L_d, \Delta', E \,;\, E_o\rangle} \tag{R-Observe}$$

Model-local derivation.

$$\frac{y\#C \qquad \Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright y := \mathsf{derive}_\tau(S)!E_y \Rightarrow \Delta'}{\Gamma;\Pi \vdash \langle R, a@L_d, \Delta, E\rangle \xrightarrow{\mathsf{derive}(q,y,S)} \langle R, a@L_d, \Delta', E\,;E_y\rangle} \quad \text{(R-Derive)}$$

Proposal.

$$\frac{p\#C \qquad \Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright p := \mathsf{propose}(\delta)!E_p \Rightarrow \Delta'}{\Gamma;\Pi \vdash \langle R, a@L_d, \Delta, E\rangle \xrightarrow{\mathsf{propose}(q,p,\delta)} \langle R, a@L_d, \Delta', E\,;E_p\rangle} \quad \text{(R-Propose)}$$

Commitment.

$$\frac{\Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright \mathsf{commit}(p,\alpha)!E_c \Rightarrow \Delta'}{\Gamma;\Pi \vdash \langle R, a@L_d, \Delta, E\rangle \xrightarrow{\mathsf{commit}(q,p,\alpha)} \langle R, a@L_d, \Delta', E\,;E_c\rangle} \quad \text{(R-Commit)}$$

Commitment appends authority state to the trace; it does not change the executable workflow state.

Tool call. The typed action and the invocation transition contain the same tool, inputs, result names, and metadata. The runtime may report any attested branch, but no other branch.

$$\frac{\begin{array}{c}\bar{y}\#C \\ \Gamma;\Pi;\Delta;E \vdash a@L_d[q] \triangleright \bar{y} := \mathsf{call}(t,x,\eta)!\mathsf{lab}_q(F_t) \Rightarrow \Delta' \\ R \xrightarrow{\mathsf{invoke}(t,x,\bar{y},\eta)/F_t} R'\end{array}}{\Gamma;\Pi \vdash \langle R, a@L_d, \Delta, E\rangle \xrightarrow{\mathsf{call}(q,t,x,\bar{y},\eta,F_t)} \langle R', a@L_d, \Delta', E\,;\mathsf{lab}_q(F_t)\rangle} \quad \text{(R-Call)}$$

The membership premise of T-Call, the universal branch checks, and the exact operational tuple together rule out metadata substitution. For every production branch, prefix admissibility already requires an earlier commitment with the same proposal, workflow, and patch; no second commitment representation is needed.

7.3. Lifting security-relevant runtime steps

A security-relevant runtime event is admitted only at derived stage execute, with the exact committed wrapper metadata, conservative dependencies, fresh output bindings, required capabilities, and an admissible lifted prefix.

$$\frac{\begin{array}{c}R \xrightarrow{\mu} R' \quad \mathsf{auth}_R(\mu) = (q,p,W,\delta) \quad \mathsf{stage}_{\Delta,E}(q) = \mathsf{execute} \quad \exists\alpha, L_c.\ (q, \mathsf{Commit}(p,W,\delta,\alpha,L_c)) \in E \\ \mathsf{deps}_R(\mu) = D_\mu \quad \mathsf{out}_R(\mu) = \bar{z} : \bar{\tau} \quad \forall i.\ \mathsf{data}(\tau_i) \quad \bar{z}\#C \\ \mathsf{lift}_{p,W,\delta}(\mu, D_\mu, \bar{z}) = F_r \quad \mathsf{binds}^{=}(F_r,\bar{z}) \quad \neg\mathsf{hasCommit}(F_r) \quad \mathsf{RuntimeUpdate}(\Gamma,\Delta,E,\mu,\Delta') \quad \Gamma \vdash a : \mathsf{Del}[\kappa] \quad \mathsf{req}(F_r) \subseteq \kappa \quad \mathsf{Adm}_{\Gamma,\Pi,a}(E\,;\mathsf{lab}_q(F_r))\end{array}}{\Gamma;\Pi \vdash \langle R, a@L_d, \Delta, E\rangle \xrightarrow{\mu} \langle R', a@L_d, \Delta', E\,;\mathsf{lab}_q(F_r)\rangle} \quad \text{(R-Runtime)}$$

The update premise makes the runtime/environment interface exact. For an execution event with outputs $\bar{z} : \bar{\tau}$, it requires $\mathsf{ExactExt}(\Delta, \Delta', [(q, z_i, \tau_i)]_i)$; for a transfer, $\bar{z} = ()$ and it requires $\Delta' = \Delta$. The exact-binding premise then makes those bindings coincide with all and only the Compute targets in the lifted fragment. The no-commit premise prevents a trusted runtime adapter from creating new authority while reporting an execution event.

By lifting completeness, events outside the lifting domain are not protection relevant; they change only the internal runtime state:

$$\frac{R \xrightarrow{\mu} R' \qquad \mu \notin \mathrm{dom}(\mathsf{lift})}{\Gamma;\Pi \vdash \langle R, a@L_d, \Delta, E\rangle \xrightarrow{\mu} \langle R', a@L_d, \Delta, E\rangle} \quad \text{(R-Internal)}$$

Equation (10) therefore reflects each admitted SWIRL execution as an exact Act plus conservative output dependencies, and each admitted send as an Expose. An inadmissible lifted fragment has no Hard-KLAIM transition; this is refusal at the enforcement boundary, not a change to the internal SWIRL semantics.

Definition 19 (Strengthened boundary transition). *The public label of a core transition records the arguments needed to audit the rule: observation, derivation, proposal, commitment, the complete tool-call tuple including its reported fragment, a runtime event, or an internal tag. A strengthened transition*

$$\Gamma;\Pi \vdash \langle C, A, H\rangle \overset{\lambda}{\Longrightarrow} \langle C', A', H'\rangle$$

holds when the corresponding core rule gives $\Gamma; \Pi \vdash C \xrightarrow{\lambda} C'$, $H' = H$; λ, *and* $A' = A$. *A strengthened transition operates in an already active context; context creation, if supplied by a deployment, is a separate administrative operation. This wrapper is deliberately exact: the history contains the label emitted by the same core transition, once and in order.*

7.4. Trace safety of reachable configurations

Having defined the operational semantics of the framework, we can now formalise properties of interest, and then state the main theorem of *trace safety.*

7.4.1. Proof-complete boundary invariants

The one-step confinement proof is organised around the following family of mutually supporting invariants: stage consistency, production consistency, exact authority, provenance, capability bounds, source-closed exposure, and boundary fidelity. Their auxiliary notions add no operational mechanism; they expose the finite state and the two-way correspondences retained by every inductive preservation step.

Definition 20 (Origins and conservative provenance). *A dynamic name has one of four boundary origins:*

$$o ::= \mathsf{model}(q) \mid \mathsf{tool}(q, t, \eta) \mid \mathsf{runtime}(q, \mu) \mid \mathsf{proposal}(q).$$

The relation $\mathsf{introduces}(\lambda, q, y, o)$*is determined by the label: derivation introduces its result with model origin; a tool call introduces each declared result with tool origin; runtime execution introduces each reported output with runtime origin; and proposal introduces its proposal identifier. The judgement* $\mathsf{origin}_H(q, y, o)$ *holds when such a label occurs in H. In a well-formed configuration every dynamic name has exactly one such origin, globally across contexts.*

For non-proposal outputs, the origin must also support a matching production in the trace. A model output includes the host-visible lower bound in its source set; a tool output includes every named call input; a runtime output uses the adapter's dependency tuple. This is conservative provenance*. It does not attempt to reconstruct the internal causal behaviour of a model, tool, or runtime; it guarantees that the trusted boundary did not omit the dependencies required by its interface.*

Definition 21 (Exact production delta). *For one transition, let B be its output bindings and F its appended labelled effects. Their exact correspondence is recorded by*

$$\mathsf{ExactProductionDelta}(\Delta, E, \Delta', E', B, F).$$

It requires:

(i) $\mathsf{ExactExt}(\Delta, \Delta', B)$ *and* $E' = E$; F*;*

(ii) every name in B is absent as a production target in E;

(iii) every binding in B has exactly one matching Compute *in F;*

(iv) every Compute *target in F has a matching binding in B.*

This bridge is required because an environment equation alone says nothing about trace production, while an emitted Compute *alone says nothing about which binding was installed. Exact tool and runtime output premises establish both directions; non-producing rules use* $B = \emptyset$ *and a fragment with no* Compute.

Definition 22 (Exact proposal–approval–commitment binding). *Exact proposal–approval–commitment binding is the conjunction of three persistent facts. First, each proposal binding has the exact provenance*

$$\mathsf{Compute}(\{W, \delta\} \Rightarrow p, L).$$

Second, each commitment retains that binding, passes

$$\mathsf{verify}_\Gamma(\alpha, a, q, p, W, \delta) = \mathsf{true},$$

and follows the proposal production. Third, each $\mathsf{Act}(p, k, W, \delta, Y, L)$ *follows a commitment with the same* (q, p, W, δ)*. This excludes both proposal substitution and approval replay under a different tuple.*

Definition 23 (Capability non-amplification). *Capability non-amplification requires, for every label in H, that the capabilities demanded by its exact emitted fragment are a subset of the fixed declaration $\Gamma \vdash a : \mathsf{Del}[\kappa]$. Direct derivation, proposal, and commitment labels demand their respective capabilities; observation, calls, and runtime labels use* req*. Dynamic values and runtime state never extend κ.*

Definition 24 (Source-closed exposure). *Source-closed exposure requires, for every decomposition*

$$E = E_0 \,;\, (q, \mathsf{Expose}(Y, L_1, L_2)) \,;\, E_1,$$

that $E_0 \Downarrow_\Gamma Y@L_1$ and, for every $B \in \mathsf{sources}_{E_0}(Y)$,

$$O_\Pi(\Gamma, E_0, \mathsf{flow}(a, B, L_1, L_2)) = \mathsf{allow}.$$

The reflexivity of source closure ensures that the carrier Y itself is checked even if it has no recorded predecessors. Quantifying over the exact decomposition ensures that later dependencies cannot be used to justify an earlier exposure.

Definition 25 (Exact boundary fidelity). *Exact boundary fidelity joins interface identity to the exact production delta. In the call case, the declaration, result shape, selected schema member, call permission, guard, exact output binding, and invocation all use the same $(t, x, \bar{y}, \eta, F_t)$. In the runtime case, the same event μ fixes the runtime transition, execute stage, committed wrapper, dependency tuple, outputs, and exact environment update. In every case the public label emits exactly the suffix used by the production delta. This is the formal reason a checked backend, output tuple, proposal identifier, or runtime event cannot be silently replaced at invocation time.*

Definition 26 (Complete well-formedness). *A strengthened configuration $\widehat{C} = \langle C, A, H\rangle$ is well formed when: A has no duplicates; the delegate and every trace locality are declared; all dynamic, trace, and history contexts are active; dynamic names are globally single-assignment and do not alias static or initially placed names; every dynamic binding has one producer and one boundary origin; proposal bindings, commitments, and actions satisfy exact authority; and every context containing an action has exactly one production-call label in H. These clauses state all representation and consistency requirements needed by one-step preservation.*

Theorem 1 (One-step provenance-and-authority confinement). *Let $\widehat{C}$ be completely well formed (Definition 26) and satisfy conservative provenance (Definition 20), capability non-amplification (Definition 23), and source-closed exposure (Definition 24). If it admits the strengthened boundary transition of Definition 19,*

$$\Gamma; \Pi \vdash \widehat{C} \overset{\lambda}{\Longrightarrow} \widehat{C}',$$

then active stages in $\widehat{C}'$ are deterministic and do not regress (Proposition 1). Dynamic bindings remain single-assignment and uniquely produced (Proposition 2 and definition 21). The target retains conservative provenance (Definition 20), exact proposal–approval–commitment binding (Definition 22), capability non-amplification (Definition 23), and source-closed exposure (Definition 24). The transition also has exact boundary fidelity (Definition 25).

Intuitively, the theorem confines one admitted boundary step in three dimensions. The protocol cannot move backwards or acquire a second production call; newly installed values are precisely the freshly recorded outputs with the appropriate origin and conservative sources; and the step can exercise only previously declared authority, including source-sensitive flow authority. The theorem constrains the public boundary record and its bindings, not the private reasoning of the LLM.

Proof justification. Each operational constructor first yields an exact production delta. The ordered stage definition and persistence of old bindings and effects give stage determinism and monotonicity. Freshness and the two directions of the delta give single assignment and unique production. The label-specific rule premises give model, tool, or runtime provenance and capability bounds. Prefix admissibility preserves source-closed exposure; the verified commitment rule and the no-commit restrictions on calls and runtime fragments preserve exact authority. Finally, inversion of the call and runtime rules identifies the checked tuple, the trusted interface transition, the emitted label, and the same exact delta. Internal steps leave every tracked component unchanged.

7.4.2. Interface adequacy, administrative closure, and finite reachability

Definition 27 (Interface adequacy). *The predicate* $\mathsf{InterfaceOK}_{\Gamma,\Pi}$ *collects the obligations owned by the trusted boundary rather than by the transition system. It holds when the locality, workflow, delegate, and tool declarations are authoritative; the visible-set construction contains every protected value visible to the model; tool schemas conservatively report their protection-relevant effects; the evidence verifier checks the exact approval tuple; the policy oracle implements* Π*; and the runtime adapter faithfully and completely reports protection-relevant events, dependencies, localities, outputs, and types. These are precisely the interface premises used by the typing and transition rules. Naming their conjunction prevents the trace theorem from hiding a distributed assumption behind an informal reference to those rules.*

Definition 28 (Administrative closure). *The one-step theorem cannot create authoritative deployment facts. For example, it cannot prove that a locality reported by an abstract adapter was declared unless the adapter supplies that fact. The residual obligation is* $\mathsf{AdminClosed}_{\Gamma}(\widehat{C})$*. It states that the active list is duplicate-free; the delegate and all mentioned localities are declared; all dynamic, trace, and history contexts are active; dynamic names do not alias static or initially placed names and have one boundary origin; and execution has one production-call label. These checks are administrative rather than security conclusions: a concrete finite representation can discharge them by declaration lookup, context ownership, and exact history maintenance.*

Definition 29 (Persistent trace invariant). *The persistent induction invariant* $\mathsf{TraceInv}_{\Gamma,\Pi}(\widehat{C})$*consists of complete well-formedness, conservative provenance, capability non-amplification, and source-closed exposure.*

Definition 30 (Administratively closed finite reachability). *An* administratively closed finite reachability *derivation is the reflexive-transitive closure of strengthened boundary steps in which every successor additionally satisfies* $\mathsf{AdminClosed}$*. This is not an assumption of the desired security properties. It is the bridge that combines deployment-owned structural facts with the one-step conclusions so that full well-formedness can be reconstructed for the next induction step.*

Theorem 2 (Trace safety). *Let*

$$\widehat{C_0} = \langle\langle R_0, a@L_d, \Delta_0, \varepsilon\rangle, A_0, \varepsilon\rangle$$

be completely well-formed in the sense of Definition 26, with $\Delta_0(q)$ *empty for every* $q \in A_0$*. Assume* $\mathsf{InterfaceOK}_{\Gamma,\Pi}$ *as specified by Definition 27. If* $\widehat{C}$ *is reachable from* $\widehat{C_0}$ *through an administratively closed finite execution in the sense of Definition 30, then* $\widehat{C}$ *is completely well-formed and has:*

(a) deterministic active protocol stages (Proposition 1);

(b) global single assignment and one unique producer for every dynamic name (Proposition 2 and definition 21);

(c) conservative provenance for every model, tool, and runtime output (Definition 20);

(d) exact proposal–approval–commitment binding for every production action (Definition 22);

(e) capability non-amplification for every recorded boundary label (Definition 23); and

(f) source-closed exposure at the exact historical prefix of every transfer (Definition 24).

Moreover, every transition in the reachability derivation has exact boundary fidelity (Definition 25) by Theorem 1.

Trace safety is stronger than an admissibility lemma: all intermediate configurations retain the premises required to check each successor step, so the endpoint facts hold for an arbitrary finite execution rather than for one isolated transition. Exact authority prevents a model-generated proposal from becoming a different committed action; conservative provenance and source closure prevent an exposure from forgetting the protected inputs that may have influenced it; and capability non-amplification prevents accumulated runtime state from silently increasing the delegate's authority.

Proof. By induction on administratively closed finite reachability. The empty trace and history make the provenance, capability-history, and exposure-history clauses vacuous at the initial configuration; assumed initial well-formedness supplies the remaining clauses. In the successor case, apply Theorem 1 to the induction invariant. Its single-assignment, unique-production, and exact-authority conclusions, together with AdminClosed for the successor, reconstruct complete well-formedness. Its provenance, capability, and exposure conclusions supply the other fields of TraceInv, which is therefore available for the next step. At the endpoint, well-formedness projects deterministic stages, single assignment, unique production, and exact authority, while the remaining three conclusions project directly from the invariant. This completes the induction.

Scope and limitations of the theorem. The theorem is a boundary-safety result, not an end-to-end verification result for an LLM, MCP implementation, workflow engine, or distributed infrastructure. It establishes that every execution represented by the Hard-KLAIM boundary semantics preserves the stated commitment, provenance, flow, and stage invariants, provided that the authoritative interface and policy are correct, tool effect schemas conservatively describe protection-relevant behaviour, the host visible set does not omit model-visible protected values, approval evidence is trustworthy, and the runtime adapter faithfully and completely reports protection-relevant execution events, dependencies, localities, outputs, and types. Under these assumptions, an LLM cannot obtain a production action in the modelled trace without the exact preceding commitment, substitute another workflow change after approval, silently change checked invocation metadata, or expose a derived value without satisfying the flow policy for its recorded sources.

The theorem does not establish that these assumptions hold in a concrete deployment. In particular, it does not verify the internal reasoning of the LLM, prove the correctness of the policy itself, derive sound effect schemas automatically from arbitrary MCP tools, or prove that a workflow runtime and its adapter report all relevant behaviour correctly. Nor does it establish liveness, availability, scientific correctness of a workflow modification, or freedom from effects that are outside the protection-relevant boundary model. A compromised component inside the trusted boundary may therefore invalidate the guarantee.

A stronger end-to-end result would progressively discharge these assumptions rather than enlarge the core calculus. One direction is to derive or verify tool-effect schemas against concrete MCP implementations; another is to establish a refinement relation between runtime semantics such as SWIRL and the Hard-KLAIM lifting, proving that every protection-relevant runtime transition is represented by the required effects. The trusted visible-set construction and policy oracle could likewise be connected to concrete enforcement mechanisms. Mechanising these refinement arguments and validating them against an enforcement prototype would turn the conditional boundary-safety theorem into a substantially stronger assurance result for deployed agentic workflows.

8. Related Work and Positioning

8.1. Tool-using LLM *systems*

MRKL, ReAct, Toolformer, AutoGen, and related agent frameworks establish the architectural pattern in which language models select and compose external tools [8, 9, 10, 11, 12]. Hard-KLAIM does not propose another agent architecture. It addresses the semantic question that arises when such a model-mediated trace is allowed to affect distributed workflow objects: which externally visible effects are admissible under locality, commitment, and information-flow constraints?

8.2. Concurrency formalisms, behavioural types, and KLAIM

Hard-KLAIM behaviours can in principle be represented in established concurrency and behavioural formalisms. The π-calculus models mobile interaction through name passing [16]; session and behavioural types constrain interaction protocols [17]; and typestate systems track which operations are valid at a program state [18]. No expressiveness-separation claim is required against these general-purpose formalisms.

The distinction is instead one of semantic abstraction. This is also the methodological connection with KLAIM: KLAIM made locality and mobility explicit semantic concepts on which interaction and access-control properties could be stated, rather than claiming that locality could not be encoded in another calculus [1, 19, 20]. Its original type-based access-control discipline approximates the operations that a process intends to perform at particular localities

and checks those intentions against the rights available at the corresponding protection domains [7]. Subsequent work made privilege acquisition dynamic, combining static and dynamic checks with reference monitoring so that capabilities can be obtained and used as the computation evolves [21]. Confinement analyses then made movement constraints explicit: annotations on data and nodes restrict where data and processes may reside or migrate [22]. Flow-Logic-based analyses further showed how the same KLAIM lineage can obtain static guarantees for tuple-space access and safe process migration [23].

Hard-KLAIM follows this line but changes both the controlled actor and the security object. The controlled actor is a model-mediated workflow delegate rather than mobile code, and the relevant decision depends on an accumulated delegation history rather than only on an operation and its locality. The effect trace therefore makes proposal identity, immutable workflow change, commitment evidence, production actions, source dependencies, and cross-locality exposure first-class. Dynamic authority is represented by an explicit proposal–commit–execute discipline, while information movement is checked against the transitive sources of derived values. More general calculi could encode these behaviours, but would require an additional representation and adequacy argument connecting that encoding to the workflow-security interpretation. Our contribution does not depend on an expressiveness-separation claim.

The methodological progression can be summarized as follows:

KLAIM: locality + mobility + access rights
↓
dynamic privileges / confinement / static security analysis
↓
Hard-KLAIM: locality + delegated authority + commitment + provenance + flow

8.3. Information flow, runtime enforcement, and workflow security

Source-sensitive exposure is related to information-flow control, which restricts the destinations to which protected information may flow [24]. Runtime enforcement and security automata monitor execution traces and suppress inadmissible actions [25], while secure service composition studies how individually acceptable services may compose into unsafe behaviour [26]. Hard-KLAIM combines these concerns at the workflow boundary: the trace records whether a workflow change is still a proposal, whether it has been committed, where execution occurs, and which sources contribute to an exposed artefact.

Scientific workflow systems such as CWL and StreamFlow supply the execution setting [2, 3, 13]; SWIRL provides the concrete distributed runtime semantics consumed by Hard-KLAIM [14]; provenance models record what happened during execution; and conventional access-control mechanisms determine whether a principal may invoke an endpoint. Hard-KLAIM targets the compositional gap between these mechanisms: whether a sequence of individually available calls is an authorised workflow transition and information flow before the corresponding production-side action is admitted.

9. Conclusion

The question that motivated KLAIM can be paraphrased as follows: what can a mobile agent safely do when it interacts with remote localities? For model-mediated workflows, the corresponding question is:

what can an LLM*-based agentic delegate safely make executable when it invokes* MCP*-exposed workflow tools connected to remote and distributed runtime infrastructures?*

Hard-KLAIM provides a compact semantic answer. By making localities, immutable workflow versions, typed tool effects, proposal identity, commitment, and cross-locality exposure explicit, Hard-KLAIM extends the locality-centred discipline of KLAIM from access control for mobile agents to delegated authority over agentic workflows. The object being controlled is no longer only mobile code, but authority exercised through an LLM, workflow-facing tools and distributed workflow runtimes.

The resulting boundary is deliberately modular. The workflow runtime determines how a distributed computation evolves, while Hard-KLAIM determines whether the security-relevant execution and transfer steps exposed at that boundary are admissible. In the concrete instantiation, SWIRL supplies the runtime semantics. Under the stated assumptions on schemas, policy information, visible values, approval evidence, and runtime adaptation, the trace-safety

theorem guarantees deterministic protocol stages, single assignment and unique production, conservative provenance, exact commitment, capability non-amplification, source-closed exposure, and per-step boundary fidelity for administratively closed reachable configurations. Such main result of this work is therefore a boundary-safety guarantee, not an end-to-end verification of the LLM, the MCP implementation, or the workflow runtime.

Revisiting KLAIM in the setting of LLM-mediated workflows showed that its central ideas remain technically relevant when computation, authority, and data once again cross explicit locality boundaries, although through very different software artefacts. The resulting research hypothesis is that hardening agentic workflows requires a semantic account of delegated authority and commitment, not merely per-tool authorisation. In this sense, Hard-KLAIM is both a continuation of the locality-centred view developed around KLAIM and a proposal for applying that view to a new form of distributed delegation.

The natural next step is to reduce the trust assumptions on which the guarantee granted by the Hard-KLAIM formalism depends. A concrete enforcement layer would connect Hard-KLAIM judgements to MCP tool metadata, immutable output identities, proposal-specific approval evidence, workflow-patch classifiers, and audit traces.

Tool-effect schemas could be derived from or verified against concrete tool implementations. The SWIRL lifting could be related to the runtime by a refinement argument showing that every protection-relevant transition is represented at the Hard-KLAIM boundary, and the visible-set and policy-oracle assumptions could be tied to concrete host, filesystem, and scheduler mechanisms. For systems such as CWL and StreamFlow, this would require classifying changes to resources, container images, parameters, data locations, scheduler directives, and scientific tool arguments, mapping them to typed effects and commitment policies, and evaluating the resulting enforcement on realistic model-mediated workflow traces.

10. Acknowledgements

The authors thank the anonymous reviewers for their careful reading and constructive comments. The work was partially supported by the European Union Horizon 2020 research and innovation programme under grant agreement No. **XXX** and by the Italian Ministry of University and Research under **XXX**.

References

[1] R. De Nicola, G.-L. Ferrari, R. Pugliese, KLAIM: A kernel language for agents interaction and mobility, IEEE Transactions on Software Engineering 24 (5) (1998) 315–330. doi:10.1109/32.685256.

[2] M. R. Crusoe, S. Abeln, A. Iosup, P. Amstutz, J. Chilton, N. Tijanic, H. Ménager, S. Soiland-Reyes, B. Gavrilovic, C. A. Goble, T. C. Community, Methods included: Standardizing computational reuse and portability with the Common Workflow Language, Communications of the ACM 65 (6) (2022) 54–63. doi:10.1145/3486897.

[3] I. Colonnelli, B. Cantalupo, I. Merelli, M. Aldinucci, StreamFlow: Cross-breeding cloud with HPC, IEEE Transactions on Emerging Topics in Computing 9 (4) (2021) 1723–1737. doi:10.1109/TETC.2020.3019202.

[4] Anthropic, Model context protocol, accessed: 1 September 2026 (2024).
URL https://modelcontextprotocol.io/

[5] N. Shapira, C. Wendler, A. Yen, G. Sarti, K. Pal, O. Floody, A. Belfki, A. Loftus, A. R. Jannali, N. Prakash, J. Cui, G. Rogers, J. Brinkmann, C. Rager, A. Zur, M. Ripa, A. Sankaranarayanan, D. Atkinson, R. Gandikota, J. Fiotto-Kaufman, E. Hwang, H. Orgad, P. S. Sahil, N. Taglicht, T. Shabtay, A. Ambus, N. Alon, S. Oron, A. Gordon-Tapiero, Y. Kaplan, V. Shwartz, T. Rott Shaham, C. Riedl, R. Mirsky, M. Sap, D. Manheim, T. Ullman, D. Bau, Agents of chaos, arXiv preprint arXiv:2602.20021 (2026). doi:10.48550/arXiv.2602.20021.

[6] S. Willison, The lethal trifecta for AI agents: Private data, untrusted content, and external communication, accessed: 1 September 2026 (2025).
URL https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/

[7] R. D. Nicola, G. Ferrari, R. Pugliese, B. Venneri, Types for access control, Theor. Comput. Sci. 240 (1) (2000) 215–254. doi:10.1016/S0304-3975(99)00232-7.

[8] E. Karpas, O. Abend, Y. Belinkov, B. Lenz, O. Lieber, N. Ratner, Y. Shoham, H. Bata, Y. Levine, K. Leyton-Brown, D. Muhlgay, N. Rozen, E. Schwartz, G. Shachaf, S. Shalev-Shwartz, A. Shashua, M. Tenenholtz, MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning, arXiv preprint arXiv:2205.00445 (2022). doi:10.48550/arXiv.2205.00445.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations, 2023, pp. 1–33. doi:10.48550/arXiv.2210.03629.
URL https://openreview.net/forum?id=WE_vluYUL-X

[10] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Thirty-seventh Conference on Neural Information Processing Systems, Vol. 36, 2023, pp. 68539–68551. doi:10.52202/075280-2997.

[11] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, C. Wang, AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework, arXiv preprint arXiv:2308.08155 (2023). doi:10.48550/arXiv.2308.08155.

[12] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, J.-R. Wen, A survey on large language model based autonomous agents, arXiv preprint arXiv:2308.11432 (2023). doi:10.1007/s11704-024-40231-1.

[13] I. Colonnelli, B. Cantalupo, R. Esposito, M. Pennisi, C. Spampinato, M. Aldinucci, HPC application cloudification: The streamflow toolkit, in: PARMA-DITAM 2021, Vol. 88 of Open Access Series in Informatics, 2021, pp. 5:1–5:13. doi:10.4230/OASIcs.PARMA-DITAM.2021.5.

[14] I. Colonnelli, D. Medić, A. Mulone, V. Bono, L. Padovani, M. Aldinucci, Introducing SWIRL: An intermediate representation language for scientific workflows, in: Formal Methods. FM 2024, Vol. 14933 of Lecture Notes in Computer Science, Springer Nature Switzerland, 2025, pp. 226–244. doi:10.1007/978-3-031-71162-6_12.

[15] Anthropic, Model context protocol, specification: Security and trust & safety, accessed: 1 September 2026 (2025).
URL https://modelcontextprotocol.io/specification/2025-03-26

[16] R. Milner, J. Parrow, D. Walker, A calculus of mobile processes, i, Information and Computation 100 (1) (1992) 1–40. doi:10.1016/0890-5401(92)90008-4.

[17] K. Honda, V. T. Vasconcelos, M. Kubo, Language primitives and type discipline for structured communication-based programming, in: Programming Languages and Systems, Vol. 1381 of Lecture Notes in Computer Science, Springer, 1998, pp. 122–138. doi:10.1007/BFb0053567.

[18] R. E. Strom, S. Yemini, Typestate: A programming language concept for enhancing software reliability, IEEE Transactions on Software Engineering SE-12 (1) (1986) 157–171. doi:10.1109/TSE.1986.6312929.

[19] R. De Nicola, G.-L. Ferrari, R. Pugliese, Programming access control: The KLAIM experience, in: CONCUR 2000 – Concurrency Theory, Vol. 1877 of Lecture Notes in Computer Science, Springer, 2000, pp. 48–65. doi:10.1007/3-540-44618-4_5.

[20] L. Bettini, V. Bono, R. De Nicola, G.-L. Ferrari, D. Gorla, M. Loreti, E. Moggi, R. Pugliese, E. Tuosto, B. Venneri, The KLAIM project: Theory and practice, in: Global Computing: Programming Environments, Languages, Security, and Analysis of Systems, Vol. 2874 of Lecture Notes in Computer Science, Springer, 2003, pp. 88–150. doi:10.1007/978-3-540-40042-4_4.

[21] D. Gorla, R. Pugliese, Resource access and mobility control with dynamic privileges acquisition, in: Automata, Languages and Programming, Vol. 2719 of Lecture Notes in Computer Science, Springer, 2003, pp. 119–132. doi:10.1007/3-540-45061-0_11.

[22] R. De Nicola, D. Gorla, R. Pugliese, Confining data and processes in global computing applications, Science of Computer Programming 63 (1) (2006) 57–87. doi:10.1016/j.scico.2005.07.013.

[23] R. De Nicola, D. Gorla, R. R. Hansen, F. Nielson, H. R. Nielson, C. W. Probst, R. Pugliese, From flow logic to static type systems for coordination languages, Science of Computer Programming 75 (6) (2010) 376–397. doi:10.1016/j.scico.2009.07.009.

[24] A. C. Myers, JFlow: Practical mostly-static information flow control, in: Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, ACM, 1999, pp. 228–241. doi:10.1145/292540.292561.

[25] F. B. Schneider, Enforceable security policies, ACM Transactions on Information and System Security 3 (1) (2000) 30–50. doi:10.1145/353323.353382.

[26] M. Bartoletti, P. Degano, G.-L. Ferrari, Enforcing secure service composition, in: 18th IEEE Computer Security Foundations Workshop, IEEE Computer Society, 2005, pp. 211–223. doi:10.1109/CSFW.2005.17.

Appendix A. Filesystem-and-Scheduler Policy Oracle

A complete, deliberately narrow oracle instantiation treats workflow inputs and outputs exposed at the boundary as files or directories, and production execution is mediated by a scheduler with finite resource limits.

Principals and locality-specific identities are deployment metadata of this oracle instance, not name classes of the core calculus. Let $u \in \mathsf{Principal}$ and $u_L \in \mathsf{LocalId}$. The instance is configured with finite partial maps

$$\mathsf{principal}_O : \mathsf{DelName} \rightharpoonup \mathsf{Principal}, \qquad \iota_O : \mathsf{Principal} \times \mathsf{LocName} \rightharpoonup \mathsf{LocalId}.$$

Thus $\mathsf{principal}_O(a) = u$ identifies the authority behind delegate a, and $\iota_O(u, L) = u_L$ resolves it to the local account, service identity, or credential whose permissions and quotas apply at L. Undefined resolution denies the corresponding query unless this policy instance supplies an explicit alternative rule. The platform also supplies terminating predicates

$$\mathsf{fsRead}(u_L, p, L), \quad \mathsf{fsWrite}(u_L, p, L), \quad \mathsf{modeAllowed}(u_L, k, L), \quad \mathsf{metaAllowed}(u_L, t, \eta, L),$$

and a finite quota

$$\mathsf{quota}(u_L, L) = \langle c_{\max}, t_{\max} \rangle.$$

These predicates abstract over POSIX permissions, ACLs, service identities, scheduler accounts, and equivalent local mechanisms.

Every rule below is evaluated at the current static environment Γ and trace prefix E. To make that dependence explicit, a conclusion of the form

$$O_{\mathsf{fs/hpc}} \vdash_E \mathsf{canX}(\bar{z})$$

abbreviates $O_{\mathsf{fs/hpc}}(\Gamma, E, \mathsf{x}(\bar{z})) = \mathsf{allow}$, with the corresponding core query $\mathsf{x}(\bar{z})$. Some rules do not inspect E, but commitment does: its replay check must be performed against exactly the prefix at which the query is issued.

Observation. For any file-like immutable object name Y, let $\mathsf{path}_\Gamma(Y, L) = p$ denote authoritative path resolution. The resolver covers both static names and fresh versioned names registered by a trusted producer; a dynamic name need not be inserted into Γ. The trusted producer registry is authoritative oracle metadata and is not writable by the delegate. For a fresh workflow-version name, the same registry may carry the policy-relevant metadata needed by resolvers such as $\mathsf{evolveAllowed}_\Gamma$, without turning the dynamic name into a mutable declaration in Γ. The observation query is

$$\frac{\mathsf{principal}_O(a) = u \quad \iota_O(u, L) = u_L \quad \mathsf{path}_\Gamma(Y, L) = p \quad \mathsf{fsRead}(u_L, p, L)}{O_{\mathsf{fs/hpc}} \vdash_E \mathsf{canObserve}(a, Y, L)} \qquad \text{(O-Read)}$$

The core admissibility relation also requires that the object is available at that locality.

Proposal. Let $\mathsf{evolveAllowed}_\Gamma(u, W, \delta, L)$ be a terminating lookup over the workflow evolution mode, patch class, principal role, and proposal locality. Then

$$\frac{\mathsf{principal}_O(a) = u \quad \mathsf{evolveAllowed}_\Gamma(u, W, \delta, L)}{O_{\mathsf{fs/hpc}} \vdash_E \mathsf{canPropose}(a, W, \delta, L)} \tag{O-Propose}$$

Thus proposal authority may differ across localities. A deployment whose proposal policy is locality independent may simply ignore the final argument.

Commitment. The evidence verifier checks a signed or otherwise unforgeable payload that contains the exact context, proposal, workflow version, and immutable patch. Let $\mathsf{unused}(\alpha, E)$ implement the deployment's replay policy.

$$\frac{\mathsf{verify}_\Gamma(\alpha, a, q, p, W, \delta) \quad \mathsf{unused}(\alpha, E)}{O_{\mathsf{fs/hpc}} \vdash_E \mathsf{canCommit}(a, q, p, W, \delta, \alpha, L)} \tag{O-Commit}$$

This is also the trusted basis of the evidence-indexed value judgement $\Gamma; \Delta; q \vdash \alpha : \mathsf{Approval}(a, q, p, W, \delta)$. An approval for another proposal or context cannot satisfy this rule.

Metadata-bound tool call. A declared tool can be invoked only when the principal has an accepted local identity and the concrete metadata instance is allowed at the tool locality. Schema attestation already follows from the well-formed tool declaration:

$$\frac{\mathsf{principal}_O(a) = u \quad \iota_O(u, L_t) = u_t \quad \mathsf{metaAllowed}(u_t, t, \eta, L_t)}{O_{\mathsf{fs/hpc}} \vdash_E \mathsf{canCall}(a, t, L_d, L_t, \eta)} \tag{O-Call}$$

The same η occurs in the typed and operational call, so this decision cannot be reused for a different backend, queue, or target.

Production action. An authoritative finite resolver maps the exact action effect to concrete filesystem and scheduler requirements:

$$\mathsf{req}_\Gamma(k, W, \delta, Y, L) = \langle I, O, c, t \rangle,$$

where I and O are finite input and output path sets and $\langle c, t \rangle$ is the requested core and wall-time pair. For update or allocation actions, unused components are empty or zero. The generic action rule is

$$\frac{\begin{array}{c}\mathsf{principal}_O(a) = u \quad \iota_O(u, L) = u_L \quad \mathsf{modeAllowed}(u_L, k, L) \\ \mathsf{req}_\Gamma(k, W, \delta, Y, L) = \langle I, O, c, t \rangle \\ \forall p \in I.\ \mathsf{fsRead}(u_L, p, L) \quad \forall p \in O.\ \mathsf{fsWrite}(u_L, \mathsf{parent}(p), L) \\ \mathsf{quota}(u_L, L) = \langle c_{\max}, t_{\max} \rangle \quad c \le c_{\max} \quad t \le t_{\max}\end{array}}{O_{\mathsf{fs/hpc}} \vdash_E \mathsf{canAct}(a, k, W, \delta, Y, L)} \tag{O-Act}$$

For $k = \mathsf{submit}$ or $\mathsf{execute}$, this specialises to ordinary workflow staging and scheduler checks. For $k = \mathsf{update}$, it checks the resolved repository or workflow-version output path. The exact W, δ arguments are those already carried by the Act effect.

Cross-locality flow. A direct file transfer from L_1 to L_2 is allowed when the effective identity can read the source and write the resolved destination:

$$\frac{\begin{array}{c}\mathsf{principal}_O(a) = u \quad \iota_O(u, L_1) = u_1 \quad \iota_O(u, L_2) = u_2 \\ \mathsf{path}_\Gamma(Y, L_1) = p_1 \quad \mathsf{path}_\Gamma(Y, L_2) = p_2 \\ \mathsf{fsRead}(u_1, p_1, L_1) \quad \mathsf{fsWrite}(u_2, \mathsf{parent}(p_2), L_2)\end{array}}{O_{\mathsf{fs/hpc}} \vdash_E \mathsf{canFlow}(a, Y, L_1, L_2)} \tag{O-Flow}$$

For a derived value, Hard-KLAIM asks this query for every member of its source closure. A deployment may add finite classification or declassification rules; without one, a source lacking a resolved permitted transfer is denied.

Decidability. Identity and path resolution, schema attestation lookup, evidence verification, metadata checks, permission checks, action resolution, scheduler lookup, quota comparison, and replay checks are assumed to terminate. All path sets, source closures, schemas, and traces are finite. Hence every query and $\mathsf{Adm}_{\Gamma,\Pi,a}(E)$ are decidable for this instance.

Appendix B. Complete Derivations of the Motivating Examples

Complete derivations connect the motivating traces to the revised rules by making explicit the static declarations, dynamic output bindings, call metadata, singleton schemas, and oracle answers used by the examples.

Appendix B.1. Common environment and notation

Let a be the delegate and assume

$$\Gamma \vdash a : \mathsf{Del}[\kappa], \qquad \{\mathsf{read}, \mathsf{derive}, \mathsf{propose}, \mathsf{commit}, \mathsf{act(submit)}, \mathsf{act(execute)}, \mathsf{expose}\} \subseteq \kappa.$$

The static environment contains

$$\Gamma \vdash \mathsf{workflow} : \mathsf{Wf}[\tau_{in}, \tau_{out}, \pi, \sigma, e], \qquad \Gamma \vdash \mathsf{variant\_calling} : \tau_{run},$$
$$\Gamma \vdash \mathsf{run\_logs} : \tau_{log}, \quad \Gamma \vdash \mathsf{run\_log} : \tau_{log}, \quad \Gamma \vdash \mathsf{dataset\_manifest} : \tau_{manifest}, \quad \Gamma \vdash \mathsf{public\_issue} : \tau_{public}.$$

For the tool declarations below, abbreviate

$$\begin{aligned} \tau_{wf} &= \mathsf{Wf}[\tau_{in}, \tau_{out}, \pi, \sigma, e], \\ \tau_{patch\text{-}in} &= \tau_{wf} \times \tau_{diag}, \\ \tau_{submit} &= \mathsf{Proposal(workflow, resource\_patch)} \times \tau_{run}, \\ \tau_{diagnostic\text{-}in} &= \tau_{public} \times \tau_{log} \times \tau_{manifest}. \end{aligned}$$

The relevant part of Γ contains the following well-formed tool declarations:

$$\begin{aligned} \Gamma \vdash \mathsf{diagnose\_local} &: \mathsf{Tool}[L_{diag}, \tau_{log}, \tau_{diag}, \widehat{E}_{diag}], \\ \Gamma \vdash \mathsf{diagnose\_remote} &: \mathsf{Tool}[L_{remote}, \tau_{log}, \tau_{diag}, \widehat{E}_{remote}], \\ \Gamma \vdash \mathsf{patch\_resources} &: \mathsf{Tool}[L_{patch}, \tau_{patch\text{-}in}, \mathsf{Patch(workflow)}, \widehat{E}_{patch}], \\ \Gamma \vdash \mathsf{patch\_cwl} &: \mathsf{Tool}[L_{patch}, \tau_{patch\text{-}in}, \mathsf{Patch(workflow)}, \widehat{E}_{cwl}], \\ \Gamma \vdash \mathsf{run\_streamflow} &: \mathsf{Tool}[L_{run}, \tau_{submit}, \mathsf{Unit}, \widehat{E}_{run}], \\ \Gamma \vdash \mathsf{import\_issue} &: \mathsf{Tool}[L_{import}, \tau_{public}, \mathsf{Unit}, \widehat{E}_{import}], \\ \Gamma \vdash \mathsf{run\_diagnostic\_workflow} &: \mathsf{Tool}[L_{diagnostic}, \tau_{diagnostic\text{-}in}, \tau_{report}, \widehat{E}_{diagnostic}], \\ \Gamma \vdash \mathsf{publish\_workflow\_report} &: \mathsf{Tool}[L_{publish}, \tau_{report}, \mathsf{Unit}, \widehat{E}_{publish}]. \end{aligned}$$

Each declaration satisfies $\mathsf{SchemaOK}_\Gamma(t)$, and every displayed tool locality is declared in Γ. The occurrence of resource_patch as a first-order type index does not declare or bind that name, so it remains fresh until the corresponding call returns it. The displayed declaration is the monomorphic instance used by this worked trace; a reusable implementation may expose the corresponding family polymorphic in the immutable patch name.

The brace-delimited multi-argument bundles in Figures 4–6 are compact surface notation. In the formal derivations below they are product values, written with angle brackets and assigned the product types just given. For a unit-returning tool, the formal action is $() := \mathsf{call}(t, x, \eta)$, where $() \in \mathsf{Name(Unit)}$; the figures omit this empty result tuple.

The name workflow lies in $\mathsf{WfName} \subseteq \mathsf{ValName}$, while the run, log, manifest, and issue identifiers are ordinary statically typed members of ValName; no artefact or resource name class is used. The immutable output names diagnosis, resource_patch, and diagnostic_report are not in the binding domain of Γ. They are introduced freshly by T-Call. Their output types are respectively τ_{diag}, Patch(workflow), and τ_{report}. Proposal identifiers are likewise introduced only by T-Propose.

Initial placement includes

$$\begin{aligned} &\mathsf{init}_\Gamma(\mathsf{workflow}, \mathsf{Workspace}), && \mathsf{init}_\Gamma(\mathsf{run\_logs}, \mathsf{HPC\_GPU}), \\ &\mathsf{init}_\Gamma(\mathsf{run\_log}, \mathsf{SecureWorkspace}), && \mathsf{init}_\Gamma(\mathsf{dataset\_manifest}, \mathsf{SecureWorkspace}), \\ &\mathsf{init}_\Gamma(\mathsf{public\_issue}, \mathsf{PublicRepo}). \end{aligned}$$

For each tool call in the derivation, let G_t be the unlabelled fragment associated with the call and let $F_t = \mathsf{lab}_q(G_t)$ be its labelled form. Its schema at the corresponding input, output name, and metadata is the singleton

$$\widehat{E_t}(x, \bar{y}, \eta) = \{G_t\}.$$

For a unit-returning call, $\bar{y} = ()$. Because its fragment has no Compute target, $\mathsf{binds}^{=}(G_t, ())$ holds exactly as required by T-Call. This convention retains the general finite-set rule while keeping the worked derivations readable. The metadata values $\eta_{diag}, \eta_{patch}, \eta_{run}, \eta_{remote}, \eta_{cloud}, \eta_{import}, \eta_{publish}$ are the exact values used by both T-Call and R-Call.

Let ε denote the empty trace and $\mathsf{lab}_q(F)$ context labelling. In each derivation, $\Delta_0(q) = \emptyset$ for its selected active context q, with no other value bindings. The required oracle answers are those induced by prefix admissibility or by the action-level proposal and call checks.

Appendix B.2. Complete derivation of `OK-TRACE`

Let $q = q_{ok}$, $L_d = \mathsf{Workspace}$, and $E_0 = \varepsilon$. Then $\mathsf{stage}_{\Delta_0, E_0}(q) = \mathsf{observe}$. Assume the successful queries

$$\begin{gathered}
\mathsf{canObserve}(a, \mathsf{run_logs}, \mathsf{HPC_GPU}),\\
\mathsf{canObserve}(a, \mathsf{workflow}, \mathsf{Workspace}),\\
\mathsf{canFlow}(a, \mathsf{run_logs}, \mathsf{HPC_GPU}, \mathsf{Workspace}),\\
\mathsf{canCall}(a, \mathsf{diagnose_local}, L_d, L_{diag}, \eta_{diag}), \quad \mathsf{canCall}(a, \mathsf{patch_resources}, L_d, L_{patch}, \eta_{patch}),\\
\mathsf{canFlow}(a, \mathsf{diagnosis}, \mathsf{HPC_GPU}, \mathsf{Workspace}),\\
\mathsf{canPropose}(a, \mathsf{workflow}, \mathsf{resource_patch}, L_d),\\
\mathsf{canCommit}(a, q, p_{res}, \mathsf{workflow}, \mathsf{resource_patch}, \mathsf{approval_token}, L_d),\\
\mathsf{canCall}(a, \mathsf{run_streamflow}, L_d, L_{run}, \eta_{run}),\\
\mathsf{canAct}(a, \mathsf{submit}, \mathsf{workflow}, \mathsf{resource_patch}, \mathsf{variant_calling}, \mathsf{HPC_GPU}).
\end{gathered}$$

Because the oracle may inspect the trace, each stated shorthand is assumed at every trace prefix that induces the corresponding query.

Step 1: remote observation. Let

$$F_1 = \mathsf{lab}_q(\mathsf{Read}(\mathsf{run\_logs}, \mathsf{HPC\_GPU}) \,;\, \mathsf{Expose}(\mathsf{run\_logs}, \mathsf{HPC\_GPU}, \mathsf{Workspace})), \qquad E_1 = E_0 \,;\, F_1.$$

Initial placement discharges the read availability condition; the read then makes the source available for the exposure. The observation and flow queries give $\mathsf{Adm}_{\Gamma,\Pi,a}(E_1)$. Hence

$$\Gamma; \Pi; \Delta_0; E_0 \vdash a@L_d[q] \triangleright \mathsf{observe}(\mathsf{run\_logs@HPC\_GPU})!F_1 \Rightarrow \Delta_0. \tag{B.1}$$

Step 2: dynamically bound diagnosis. Let

$$\begin{aligned}
G_2 &= \mathsf{Read}(\mathsf{run_logs}, \mathsf{HPC_GPU}) \,;\, \mathsf{Compute}(\{\mathsf{run_logs}\} \Rightarrow \mathsf{diagnosis}, \mathsf{HPC_GPU}),\\
F_2 &= \mathsf{lab}_q(G_2), \qquad E_2 = E_1 \,;\, F_2, \qquad \Delta_2 = \Delta_0[q, \mathsf{diagnosis} \mapsto \tau_{diag}].
\end{aligned}$$

The input has type τ_{log}, the output name is fresh and has the declared result shape, and the source is available at the computation locality. Moreover, $\mathsf{binds}^{=}(G_2, \mathsf{diagnosis})$ and $\mathsf{conservative}(\mathsf{run\_logs}, G_2)$ hold. The fragment has no Act, no Commit, and no proposal identifier, and the derived stage is observe. Therefore T-Call gives

$$\Gamma; \Pi; \Delta_0; E_1 \vdash a@L_d[q] \triangleright \mathsf{diagnosis} := \mathsf{call}(\mathsf{diagnose\_local}, \mathsf{run\_logs}, \eta_{diag})!F_2 \Rightarrow \Delta_2. \tag{B.2}$$

Step 3: dynamically bound immutable patch. Let

$$\begin{aligned} G_3 &= \mathsf{Read}(\mathsf{workflow}, \mathsf{Workspace})\ ;\ \mathsf{Expose}(\mathsf{diagnosis}, \mathsf{HPC\_GPU}, \mathsf{Workspace}) \\ &\quad ;\ \mathsf{Compute}(\{\mathsf{workflow}, \mathsf{diagnosis}\} \Rightarrow \mathsf{resource\_patch}, \mathsf{Workspace}), \\ F_3 &= \mathsf{lab}_q(G_3), \qquad E_3 = E_2\ ;\ F_3, \\ \Delta_3 &= \Delta_2[q, \mathsf{resource\_patch} \mapsto \mathsf{Patch}(\mathsf{workflow})]. \end{aligned}$$

The flow obligations include both diagnosis and its transitive source run_logs. After the exposure, both inputs of the final Compute are available at Workspace. The product input has type $\tau_{\textit{patch-in}}$, and the exact-binding and conservative-source conditions hold because resource_patch is the sole Compute target and its source set contains both input names. Hence

$$\Gamma; \Pi; \Delta_2; E_2 \vdash a@L_d[q] \triangleright \mathsf{resource\_patch} := \mathsf{call}(\mathsf{patch\_resources}, \langle \mathsf{workflow}, \mathsf{diagnosis} \rangle, \eta_{\textit{patch}}) \; !F_3 \Rightarrow \Delta_3. \tag{B.3}$$

Step 4: proposal. Let

$$F_4 = (q, \mathsf{Compute}(\{\mathsf{workflow}, \mathsf{resource\_patch}\} \Rightarrow p_{res}, L_d)), \quad E_4 = E_3\ ;\ F_4,$$

$$\Delta_4 = \Delta_3[q, p_{res} \mapsto \mathsf{Proposal}(\mathsf{workflow}, \mathsf{resource\_patch})].$$

The name p_{res} is fresh, both inputs are available at L_d, and the proposal query is allowed. Consequently the Compute fragment is admissible and T-Propose gives

$$\Gamma; \Pi; \Delta_3; E_3 \vdash a@L_d[q] \triangleright p_{res} := \mathsf{propose}(\mathsf{resource\_patch})!F_4 \Rightarrow \Delta_4. \tag{B.4}$$

Step 5: proposal-specific commitment. After proposal, the authoritative approval service establishes

$$\Gamma; \Delta_4; q \vdash \mathsf{approval\_token} : \mathsf{Approval}(a, q, p_{res}, \mathsf{workflow}, \mathsf{resource\_patch}),$$

as well as

$$\mathsf{verify}_\Gamma(\mathsf{approval\_token}, a, q, p_{res}, \mathsf{workflow}, \mathsf{resource\_patch}) = \mathsf{true}, \qquad \mathsf{unused}(\mathsf{approval\_token}, E_4).$$

Moreover, the proposal fragment F_4 gives $\mathsf{proposed}_{E_4}(q, p_{res}, \mathsf{workflow}, \mathsf{resource\_patch})$, and $\mathsf{stage}_{\Delta_4, E_4}(q) = \mathsf{propose}$. Let

$$F_5 = (q, \mathsf{Commit}(p_{res}, \mathsf{workflow}, \mathsf{resource\_patch}, \mathsf{approval\_token}, L_d)), \quad E_5 = E_4\ ;\ F_5.$$

Then T-Commit yields

$$\Gamma; \Pi; \Delta_4; E_4 \vdash a@L_d[q] \triangleright \mathsf{commit}(p_{res}, \mathsf{approval\_token})!F_5 \Rightarrow \Delta_4. \tag{B.5}$$

Step 6: exact production action. Let

$$\begin{aligned} G_6 &= \mathsf{Act}(p_{res}, \mathsf{submit}, \mathsf{workflow}, \mathsf{resource\_patch}, \mathsf{variant\_calling}, \mathsf{HPC\_GPU}), \\ F_6 &= \mathsf{lab}_q(G_6), \qquad E_6 = E_5\ ;\ F_6. \end{aligned}$$

The proposal identifier occurs explicitly in the input, the fragment contains no commitment, $\mathsf{stage}_{\Delta_4, E_5}(q) = \mathsf{commit}$, and the exact matching commit precedes the Act. The input has type $\tau_{\textit{submit}}$, the output tuple is the unique member of Name(Unit), and $\mathsf{binds}^{=}(G_6, ())$ holds. The exact action query is allowed. Therefore

$$\Gamma; \Pi; \Delta_4; E_5 \vdash a@L_d[q] \triangleright () := \mathsf{call}(\mathsf{run\_streamflow}, \langle p_{res}, \mathsf{variant\_calling} \rangle, \eta_{\textit{run}}) \; !F_6 \Rightarrow \Delta_4. \tag{B.6}$$

Repeated application of T-Seq to Equations (B.1)–(B.6) derives the complete `OK-TRACE`. The derivation uses no predeclared output slot and no post-hoc realises assumption.

Appendix B.3. Failed derivation of `NOT-OK-TRACE`

Let $q = q_{bad}$, $L_d = \mathsf{Workspace}$, and begin with the same safe remote observation as Step 1. The second call has the singleton fragment

$$\begin{aligned} G_{remote} &= \mathsf{Read}(\mathsf{run\_logs}, \mathsf{HPC\_GPU}) \,;\, \mathsf{Expose}(\mathsf{run\_logs}, \mathsf{HPC\_GPU}, \mathsf{LLM\_Remote}) \\ &\quad ;\, \mathsf{Compute}(\{\mathsf{run\_logs}\} \Rightarrow \mathsf{diagnosis}, \mathsf{LLM\_Remote}), \\ F_{remote} &= \mathsf{lab}_q(G_{remote}). \end{aligned}$$

At the exposure inside F_{remote}, the exact preceding prefix is

$$E^{pre}_{remote} = E_1 \,;\, (q, \mathsf{Read}(\mathsf{run\_logs}, \mathsf{HPC\_GPU})).$$

Because source closure is reflexive, $\mathsf{run\_logs} \in \mathsf{sources}_{E^{pre}_{remote}}(\mathsf{run\_logs})$. If

$$O_\Pi(\Gamma, E^{pre}_{remote}, \mathsf{flow}(a, \mathsf{run\_logs}, \mathsf{HPC\_GPU}, \mathsf{LLM\_Remote})) = \mathsf{deny},$$

then $\neg\mathsf{Adm}_{\Gamma,\Pi,a}(E_1 \,;\, F_{remote})$. The universal premise of T-Call is false, so no typing derivation exists for the remote diagnostic call.

For independence, suppose a counterfactual policy admits that flow and the trace reaches derived stage propose with

$$\Delta(q)(p_{bad}) = \mathsf{Proposal}(\mathsf{workflow}, \mathsf{resource\_patch}).$$

The candidate unit-returning call is $() := \mathsf{call}(\mathsf{run\_streamflow}, \langle p_{bad}, \mathsf{variant\_calling}\rangle, \eta_{cloud})$; its input has type τ_{submit}. Its final singleton fragment is

$$G_{cloud} = \mathsf{Act}(p_{bad}, \mathsf{submit}, \mathsf{workflow}, \mathsf{resource\_patch}, \mathsf{variant\_calling}, \mathsf{Cloud\_GPU}), \qquad F_{cloud} = \mathsf{lab}_q(G_{cloud}).$$

It necessarily fails two distinct premises: prefix admissibility finds no earlier exact commitment, and its Act branch requires derived stage commit, not propose. Independently, a deployment may also deny

$$\mathsf{canAct}(a, \mathsf{submit}, \mathsf{workflow}, \mathsf{resource\_patch}, \mathsf{variant\_calling}, \mathsf{Cloud\_GPU}).$$

Appendix B.4. Failed derivation of `NOT-OK-FLOW`

Let $q = q_{leak}$, $L_d = \mathsf{SecureWorkspace}$, and $E_0 = \varepsilon$. Thus $\mathsf{stage}_{\Delta_0,E_0}(q) = \mathsf{observe}$. Assume the oracle permits the read and inward transfer of public_issue, the two local observations, and all call-level queries, including the publication call. The only denied obligations are precisely the stated source-flow queries.

Writing the unit-returning import action formally as $() := \mathsf{call}(\mathsf{import\_issue}, \mathsf{public\_issue}, \eta_{import})$, the first three actions produce the admissible prefix

$$\begin{aligned} E_3 = (&q, \mathsf{Read}(\mathsf{public\_issue}, \mathsf{PublicRepo})) \\ &;\, (q, \mathsf{Expose}(\mathsf{public\_issue}, \mathsf{PublicRepo}, \mathsf{SecureWorkspace})) \\ &;\, (q, \mathsf{Read}(\mathsf{run\_log}, \mathsf{SecureWorkspace})) \\ &;\, (q, \mathsf{Read}(\mathsf{dataset\_manifest}, \mathsf{SecureWorkspace})). \end{aligned}$$

The import exposure is available because the public issue is initially at PublicRepo; the two observations are local.

The fourth call uses the product input $\langle \mathsf{public\_issue}, \mathsf{run\_log}, \mathsf{dataset\_manifest}\rangle : \tau_{diagnostic\text{-}in}$ and binds a fresh output:

$$F_4 = (q, \mathsf{Compute}(\{\mathsf{public\_issue}, \mathsf{run\_log}, \mathsf{dataset\_manifest}\} \Rightarrow \mathsf{diagnostic\_report}, \mathsf{SecureWorkspace})),$$

$$E_4 = E_3 \,;\, F_4, \qquad \Delta_4 = \Delta_0[q, \mathsf{diagnostic\_report} \mapsto \tau_{report}].$$

All sources are available at the computation locality, so the call is typable. Definition 12 gives

$$\mathsf{run\_log}, \mathsf{dataset\_manifest} \in \mathsf{sources}_{E_4}(\mathsf{diagnostic\_report}).$$

The candidate publication action is the unit-returning call $() := \mathsf{call}(\mathsf{publish\_workflow\_report}, \mathsf{diagnostic\_report}, \eta_{publish})$. It is argument-typed because the report is in $\Delta_4(q)$, but its singleton fragment is

$$F_5 = (q, \mathsf{Expose}(\mathsf{diagnostic\_report}, \mathsf{SecureWorkspace}, \mathsf{PublicRepo})).$$

All other flow queries induced by this exposure, including those for diagnostic_report itself and public_issue, are assumed to return allow. If either

$$O_\Pi(\Gamma, E_4, \mathsf{flow}(a, \mathsf{run\_log}, \mathsf{SecureWorkspace}, \mathsf{PublicRepo})) = \mathsf{deny} \quad \text{or}$$
$$O_\Pi(\Gamma, E_4, \mathsf{flow}(a, \mathsf{dataset\_manifest}, \mathsf{SecureWorkspace}, \mathsf{PublicRepo})) = \mathsf{deny}$$

holds, then $\neg \mathsf{Adm}_{\Gamma,\Pi,a}(E_4 \,;\, F_5)$, and the final T-Call premise fails.

Appendix B.5. What the derivations establish

The accepted trace is derivable using the same immutable values, metadata, derived-stage checks, effect schemas, and prefix-admissibility relation used by the operational theorem. The rejected traces fail at explicit premises: unauthorised source flow, missing exact commitment, and wrong stage; the cloud target may also be independently inadmissible under the deployment policy. The worked cases therefore exercise the repaired parts of the calculus rather than relying on static result slots, implicit call metadata, or an external proposal-content equality assumption.

Appendix C. Notation map

The notation map records the symbols of the core calculus and the deliberate distinctions between similar-looking forms. It is a reading aid and introduces no additional syntax or semantic state.

Appendix C.1. Names, types, and effects

Table C.2 gives the main name classes and semantic categories used throughout the calculus.

Appendix C.2. Arrows and transition symbols

Table C.3 summarizes the arrow and transition symbols; the surrounding prose explains why the similar-looking arrows are not interchangeable.

The arrows are not interchangeable: a function type, a protocol stage change, a source dependency, and a runtime transition have different domains and different proof obligations. The overloaded use of $\Rightarrow$ is limited to two closely related forms: source-to-target production inside Compute and the output-state separator in the typing judgement. All ordinary sequencing uses the single operator ;: it sequences actions in a program and effects in a trace.

Appendix C.3. Relations and structural operators

The remaining judgement and structural operators are collected in Table C.4.

The calculus deliberately keeps | for grammar alternatives and for the parallel-composition notation inherited from the KLAIM background fragment; the surrounding grammar or process-expression context disambiguates these uses. No additional arrow or name class is needed for either case.

Table C.2: Names and semantic categories.

Notation	Category	Meaning
L	locality name	Protection domain or execution locality.
W	workflow name	Immutable workflow version.
p	proposal name	Fresh identifier binding one workflow version to one proposed change.
δ	change value	Immutable patch or other proposed workflow change.
α	approval evidence	Evidence authorising one exact proposal, workflow version, and change.
a	delegate name	Subject whose actions are checked.
t	tool name	Declared, locality-hosted tool interface.
v, Y	value names	Ordinary protection-relevant values; their types distinguish artefacts, resources, reports, and so on.
r, s	action arguments	Protection-relevant values supplied to production actions.
k	action kind	Production operation such as submit, execute, update, or allocate.
q	context name	Independent delegation run.
A	active-context list	Finite duplicate-free carrier distinguishing active empty contexts from inactive ones.
H	boundary history	Ordered public labels recording which trusted rule introduced each dynamic name.
o	value origin	Model, tool, runtime, or proposal origin associated with one boundary label.
τ, β	value types	Structured value types and deployment-specific base types.
D	declaration descriptor	Static interface entry, such as a locality, value, delegate, or tool descriptor.
κ	capability set	Operations available to a delegate.
ϵ	primitive effect	One read, computation, commitment, action, or exposure event.
F	effect fragment	Finite unlabelled sequence of primitive effects.
E	effect trace	Context-labelled sequence of effects.
$\widehat{E_t}$	effect schema	Finite set of possible fragments for an exact tool invocation.
η	invocation metadata	Complete deployment-specific call metadata.

Table C.3: Arrow and transition notation.

Symbol	Use	Meaning
$\to$	function and tool type	Maps an input type to an output type.
$\longrightarrow$	protocol or workflow progression	A named monotone stage or architectural pipeline.
$\Rightarrow$	dependency and judgement result	In Compute, records source production; at the end of a typing judgement, separates effects from the resulting Δ.
$\triangleright$	action judgement	Model-mediated execution of an action or action sequence.
$\xrightarrow{\lambda}$	operational transition	One runtime step labelled by λ; $\xrightarrow{*}$ is its reflexive transitive closure.
$\rightsquigarrow_E$	dependency relation	Direct source-to-result dependency induced by a trace.
$\rightharpoonup$	partial map	A map that may be undefined, used for dynamic environments and oracle resolution.
$\mapsto$	binding/update	Associates a name with a type or updates one context component.

Table C.4: Judgement and structural notation.

Symbol	Use	Meaning
$\vdash$	typing and oracle judgement	The environment or oracle validates the proposition to its right.
$\mathsf{Adm}_{\Gamma,\Pi,a}(E)$	admissibility	The complete trace satisfies the active policy.
$\subseteq$	inclusion	Capability, dependency, or finite-set inclusion.
#	freshness	Names have not occurred in the relevant static, dynamic, or trace state.
ε	empty trace	The empty effect history.
$\mathsf{lab}_q(F)$	context labelling	Attaches context q to every effect in an unlabelled fragment.
$\Downarrow_\Gamma$	availability	A value is available at a locality in a trace prefix.
$\oplus_q$	dynamic extension	Extends the bindings for context q.
ExactExt	exact dynamic extension	Preserves all old bindings and adds all and only a finite list of fresh bindings.
$\mathsf{binds}^{=}$	exact result binding	Equates declared result names with the targets of the fragment's producing effects.
$\prec_E$	trace order	One labelled effect occurs earlier than another in E.
$\mathcal{P}_{\mathsf{fin}}$	finite powerset	Finite set of possible effect fragments.